Programação Inteira

Modelagem com variáveis inteiras e binárias

Apostila de estudo para o notebook

`Aula_Programacao_Inteira_Modelagem.ipynb`

GCC118 – Programação Matemática

Universidade Federal de Lavras (UFLA), semestre 2026/2

Material de apoio elaborado a partir das aulas da Profa. Andreza C. Beezão Moreira e do Prof. Mayron César O. Moreira e dos slides *Programação inteira – Modelagem*

Outubro de 2026

Sumário

Como usar esta apostila **1**

I Fundamentos de programação inteira **3**

1 Da programação linear à programação inteira **4**

1.1 Relembrando: o que é um problema de programação linear 4

1.2 Quando as variáveis precisam ser inteiras 5

1.3 As classes de problemas 5

1.4 Um primeiro modelo inteiro, do zero 6

1.5 Um pouco de história 6

Exercícios de fixação 7

2 Por que não basta arredondar **8**

2.1 O exemplo de Wolsey 8

2.2 Outro exemplo clássico: o livro do MIT 9

2.3 O caso binário é ainda pior 9

2.4 Quando a inteireza é natural: escala de enfermeiras 10

Exercícios de fixação 11

3 Relaxação linear, limitantes e qualidade de um modelo **12**

3.1 Relaxação linear 12

3.2 Limitantes e *gap* 12

3.3 *Branch-and-bound* em uma página 13

3.4 Formulações: o mesmo problema, modelos diferentes 14

Exercícios de fixação 15

4 Como formular modelos com variáveis inteiras **16**

4.1 O "algoritmo" de formulação de Wolsey 16

4.2 Variáveis binárias como indicadores de eventos 17

4.3 A técnica do *big-M* 17

4.4 Como escolher o valor de M 18

4.5 Lógica com variáveis binárias 19

Exercícios de fixação 20

5 PuLP: do modelo no papel ao código **21**
5.1 Anatomia de um modelo . . . 21
5.2 Índices, somatórios e dicionários . . . 22
5.3 Armadilhas comuns em PuLP . . . 23
5.4 As funções auxiliares do notebook . . . 23
Exercício de fixação . . . 24

II Resolvendo o notebook passo a passo **25**

6 Bloco 0 – Por que programação inteira? (aquecimento) **26**
6.1 0.1 – Exercício: o modelo de Wolsey em PuLP . . . 26
6.2 0.1 (continuação) – O caso binário . . . 27
6.3 0.2 – Escala de enfermeiras . . . 27
6.4 O mapa do resto da aula . . . 28

7 Bloco 1 – Implicações *se–então* **29**
7.1 1A – Custo fixo: fabricar exige pagar a preparação . . . 29
7.2 Experimento: o valor de M importa? . . . 31
7.3 1B – Produção condicionada . . . 32

8 Bloco 2 – Restrições disjuntivas **33**
8.1 2A – Modos de operação . . . 33
8.2 2B – Escalonamento em uma máquina . . . 35

9 Bloco 3 – Relações lógicas **39**
9.1 3A – Laboratório da tabela-verdade . . . 39
9.2 3B – Carteira de investimentos . . . 40
9.3 Experimento: agregada × desagregada . . . 41

10 Bloco 4 – Representação de valores discretos **43**
10.1 O problema: diâmetros de tubulação . . . 43
10.2 Construindo o modelo . . . 44
10.3 Resultado e interpretação . . . 44

11 Bloco 5 – "Ou exclusivo" e indicadores (TVL) **46**
11.1 A receita da teoria de variáveis lógicas (TVL) . . . 46
11.2 5A – Zona de ressonância . . . 47
11.3 Experimento: sem a volta . . . 48
11.4 5B (desafio) – Desconto por faixa, onde a volta *não* é necessária . . . 48
11.5 5C – O produto de binárias $\delta_3 = \delta_1\delta_2$ . . . 49

12 Bloco 6 – Problema da mochila binária **51**

12.1 Quatro versões do problema 51
12.2 O problema do notebook: carteira de projetos 51
12.3 Comparando quatro respostas 52
12.4 6B – Mochila com regras de negócio 53

13 Bloco 7 – *Bin packing* **54**
13.1 O modelo 54
13.2 Resultado: 6 caixas 55
13.3 Reflexões: relaxação fraca e simetria 56

14 Quadro-resumo: as receitas de linearização **57**

A Respostas dos exercícios de fixação **59**

B Gabarito de valores ótimos do notebook **61**

Referências **62**

Como usar esta apostila

Esta apostila foi escrita para quem está tendo o **primeiro contato com programação inteira**. Supomos apenas que você já viu o básico de programação linear (variáveis, função objetivo, restrições, região viável) e que sabe programar em Python. Tudo o mais é construído aqui, com calma e com exemplos.

O objetivo é que, ao final, você consiga resolver sozinho(a) todos os exercícios do notebook `Aula_Programacao_Inteira_Modelagem.ipynb` e, mais importante, *entenda por que* cada modelo é escrito do jeito que é.

Organização

Parte I – Fundamentos. O que é programação inteira, por que ela é necessária, por que "resolver como PL e arredondar" não funciona, o que é relaxação linear, como se formula um modelo, o que é o famoso *big-M*, um pouco de lógica com variáveis binárias e um tutorial de PuLP (a biblioteca Python usada no notebook).

Parte II – O notebook passo a passo. Um capítulo por bloco do notebook (blocos 0 a 7). Em cada um: o problema, as respostas às perguntas da discussão, a construção do modelo matemático, a tradução para código, a interpretação do resultado e os erros mais comuns.

Apêndice. Respostas dos exercícios de fixação e tabela com os valores ótimos esperados.

Como o notebook está organizado

Cada bloco do notebook segue sempre a mesma dinâmica:

Discussão → **Modelo** (no quadro) → **Exercício** (você completa os `# TODO`) → **Verificação**

As células de **Verificação** comparam o seu resultado com o gabarito *sem revelar o valor*: aparece "correto" ou "não bate com o gabarito" (às vezes com uma dica). As células de **Solução de referência** ficam ocultas; rodá-las sobrescreve a sua versão. Sugestão: só abra a solução depois de tentar, e use esta apostila como "professor particular" quando travar.

Caixas usadas no texto

Definição: conceito formal

Definições que você precisa saber de cor.

Exemplo prático: exemplo resolvido

Pequenos problemas resolvidos do início ao fim, com todos os passos.

Receita: técnica de modelagem

Uma "fórmula" pronta para transformar uma regra em restrições lineares.

Cuidado: erros comuns

Armadilhas em que quase todo mundo cai na primeira vez.

Outros autores: outras fontes

Complementos de livros, artigos, cursos abertos e documentação de solvers, sempre com a referência (lista completa no fim da apostila).

Fontes

A apostila segue o notebook da disciplina [1] e os slides *Programação inteira – Modelagem* [2], que por sua vez se baseiam em Wolsey [3], Arenales et al. [4] e Sierksma [5]. Para ampliar a visão, incluímos material de outros autores, como o capítulo de programação inteira do livro do MIT de Bradley, Hax e Magnanti [6], artigos históricos (Land e Doig, Gomory, Dantzig, Smith), surveys e a documentação de solvers comerciais. Todas as fontes estão nas Referências.

Roteiro de estudo sugerido

1. Leia os Capítulos 1 a 4 antes da aula (ou antes de abrir o notebook).
2. Faça os exercícios de fixação do fim de cada capítulo e confira no Apêndice.
3. Leia o Capítulo 5 (PuLP) com o notebook aberto e rode as duas primeiras células.
4. Para cada bloco do notebook: tente sozinho(a); se travar, leia a seção correspondente da Parte II até o ponto em que destravar, e volte ao código.
5. No fim, revise o quadro-resumo do Capítulo 14.

Parte I

Fundamentos de programação inteira

Capítulo 1

Da programação linear à programação inteira

> *"A arte de formular programas lineares e lineares inteiros é, bem, uma arte: é difícil de ensinar e ainda mais difícil de aprender."*
>
> — G. G. Brown e R. F. Dell [14], citados em [17]

A boa notícia é que essa arte tem técnicas que se repetem. O objetivo desta apostila é apresentá-las uma a uma, com exemplos.

1.1 Relembrando: o que é um problema de programação linear

Um problema de **programação linear** (PL) tem três ingredientes:

1. **Variáveis de decisão**: os números que queremos escolher (quanto produzir, quanto transportar…).
2. **Função objetivo**: uma expressão *linear* nas variáveis que queremos maximizar ou minimizar.
3. **Restrições**: desigualdades ou igualdades *lineares* que as variáveis precisam respeitar.

"Linear" significa que cada variável aparece apenas multiplicada por uma constante e somada às outras: $3x_1 + 2x_2$ é linear; x_1x_2, x_1^2, $\sqrt{x_1}$ e $\max\{x_1,x_2\}$ não são.

Exemplo prático: produção de fósforos (slides)

Uma empresa produz caixas de fósforos longos (lucro 3 por caixa) e curtos (lucro 2), em unidades de 100 000 caixas. Sejam x_1 e x_2 as quantidades produzidas.

$$\begin{aligned} \max\ & 3x_1 + 2x_2 \\ \text{s.a.}\ & x_1 + x_2 \le 9 && \text{(capacidade da máquina)} \\ & 3x_1 + x_2 \le 18 && \text{(madeira disponível)} \\ & x_1 \le 7, \quad x_2 \le 6 && \text{(caixas disponíveis)} \\ & x_1, x_2 \ge 0. \end{aligned}$$

Os pontos que satisfazem todas as restrições formam a **região viável**, um polígono convexo. O teorema fundamental da PL garante que, se existe ótimo, existe um ótimo em um **vértice** desse polígono, e é isso que o método simplex explora. Os vértices aqui são (0,0), (6,0), (4,5; 4,5), (3,6) e (0,6), com valores 0, 18, 22,5, 21 e 12. O ótimo é $x_1 = x_2 = 4{,}5$, ou seja, **450 000 caixas de cada tipo**.

O valor fracionário é um problema? Não: a unidade é "100 000 caixas", então 4,5 significa 450 000 caixas. Variáveis contínuas são uma *aproximação realista* aqui.

1.2 Quando as variáveis precisam ser inteiras

Agora imagine que a variável é o número de aviões que uma companhia compra, de enfermeiras escaladas para um turno ou de caminhões enviados a um porto. Não existe "3,7 aviões". Land e Doig, no artigo que deu origem ao método *branch-and-bound*, já diziam que problemas com variáveis inteiras surgem sempre que há **indivisibilidades**.

Há dois motivos bem diferentes para usar variáveis inteiras, e vale a pena separá-los desde já:

1. **Natureza do problema**: a quantidade é indivisível (aviões, pessoas, veículos). É o "motivo óbvio".
2. **Artifício de modelagem**: usamos variáveis *binárias* (que só valem 0 ou 1) para representar **decisões** (abrir ou não uma fábrica) e **lógica** ("se…então", "ou", "no máximo k", custo fixo…) dentro de um modelo que continua linear. É o **foco do notebook** a partir do Bloco 1.

Outros autores: a mesma distinção no livro do MIT

Bradley, Hax e Magnanti [6, Seç. 9.2] fazem a mesma separação. De um lado, a inteireza "pela natureza do problema", com o exemplo de decidir quantos porta-aviões nucleares uma marinha deve ter: a resposta é da ordem de um ou dois, e uma fração não teria significado. De outro, variáveis inteiras que surgem de (i) *condições lógicas* ("se um novo produto for desenvolvido, então uma nova fábrica deve ser construída") e (ii) *não linearidades* como custos fixos. Os autores chamam as binárias de variáveis "sim/não" (*go–no-go*), típicas de decisões estratégicas de alto custo, como investimentos.

1.3 As classes de problemas

Definição: tipos de problemas inteiros

Seja x um vetor de variáveis contínuas e y um vetor de variáveis inteiras.

- **Programação linear inteira mista (PLIM)**: há variáveis contínuas *e* inteiras.

$$\max\ cx + dy \quad \text{s.a.} \quad Ax + Gy \leq b,\ x \geq 0,\ y \geq 0 \text{ e inteiro.}$$

- **Programação linear inteira (PLI)**: todas as variáveis são inteiras.

$$\max\ cx \quad \text{s.a.} \quad Ax \leq b,\ x \in \mathbb{Z}^n_+.$$

- **Programação binária (PB ou 0–1)**: todas as variáveis são binárias.

$$\max\ cx \quad \text{s.a.} \quad Ax \leq b,\ x \in \{0,1\}^n.$$

Uma família muito importante de problemas é a de **otimização combinatória**: temos um conjunto finito $N = \{1, \ldots, n\}$ de "elementos" (itens, arestas, tarefas…), cada um com um valor c_j, e uma família $\mathcal{F}$ de subconjuntos *viáveis* de N. Queremos o melhor subconjunto viável:

$$\max_{S \subseteq N} \Big\{ \sum_{j \in S} c_j \ :\ S \in \mathcal{F} \Big\}.$$

A mochila (escolher itens que cabem), o caixeiro viajante (escolher arestas que formam um ciclo) e o *bin packing* são exemplos. Quase sempre eles são escritos como **programas binários**: $x_j = 1$ se $j \in S$.

Cuidado: "finito" não quer dizer "fácil"

Com n itens, há 2^n subconjuntos. Para $n = 60$, são mais de 10^{18} possibilidades: enumerar todas, mesmo a um bilhão por segundo, levaria mais de 30 anos. Por isso precisamos de bons modelos e de algoritmos como o *branch-and-bound* (Seção 3.3).

1.4 Um primeiro modelo inteiro, do zero

Exemplo prático: aluguel de veículos para uma excursão

Um centro acadêmico precisa transportar **50 pessoas**. Pode alugar vans (12 lugares, R$ 250 cada) e ônibus (30 lugares, R$ 600 cada). Quantos de cada para gastar o mínimo?
Passo 1 – variáveis. v = número de vans e o = número de ônibus. Não se aluga meia van: $v, o \in \mathbb{Z}_+$.
Passo 2 – restrições. Os lugares precisam bastar: $12v + 30o \geq 50$.
Passo 3 – objetivo. min $250v + 600o$.
Passo 4 – resolver. Se ignorarmos a inteireza (PL), o ônibus custa $600/30 = 20$ por lugar e a van $250/12 \approx 20{,}83$; a PL usa só ônibus: $o = 50/30 \approx 1{,}67$, custo 1000. Como o problema é pequeno, podemos enumerar as opções inteiras com o mínimo de vans para cada número de ônibus:

o	menor v com $12v \geq 50 - 30o$	lugares	custo
0	5	60	1250
1	2	54	**1100**
2	0	60	1200

O ótimo inteiro é **1 ônibus e 2 vans (R$ 1100)**. Repare que "arredondar a PL" ($o = 1{,}67 \to 2$) dá R$ 1200: é viável, mas *não é ótimo*. E o valor da PL (1000) é menor que o ótimo inteiro (1100), porque a PL tem mais liberdade. Essas duas observações são o tema dos próximos dois capítulos.

1.5 Um pouco de história

Os métodos que o solver do notebook usa têm mais de 60 anos, e vários dos problemas desta apostila são, eles mesmos, marcos históricos.

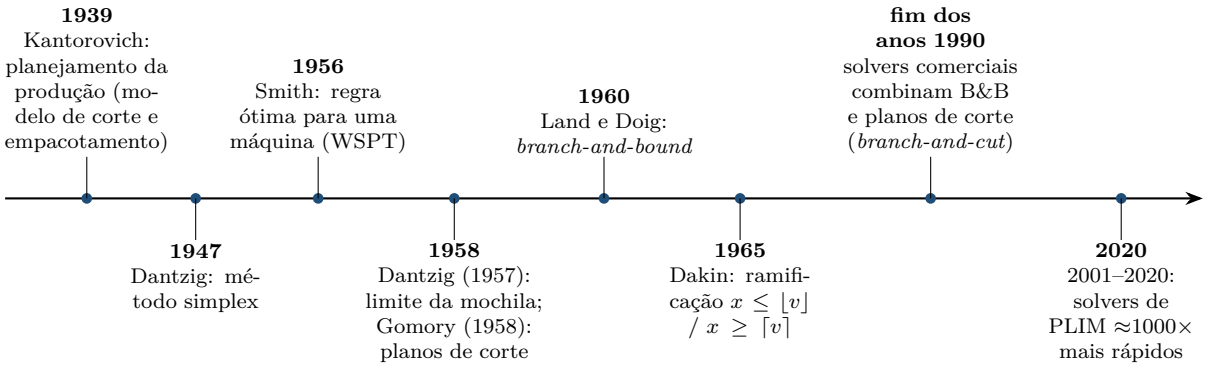


Figura 1.1: Alguns marcos da programação inteira citados nesta apostila. Fontes: Kantorovich [24]; Smith [21]; Dantzig [22]; Gomory [9]; Land e Doig [7]; Dakin [10]; Bixby [11]; Koch et al. [12]. (Posições fora de escala.)

Outros autores: Ailsa Land e Alison Doig

O *branch-and-bound* nasceu de um trabalho contratado pela British Petroleum na London School of Economics, para melhorar modelos de PL de refinarias em que algumas variáveis precisavam ser inteiras. O artigo de Land e Doig saiu na *Econometrica* em 1960 [7]. Ailsa Land foi a primeira mulher a ser professora titular de Pesquisa Operacional na Grã-Bretanha [8]. A ideia de resolver relaxações lineares e dividir o problema em subproblemas ainda é a espinha dorsal dos solvers modernos.

Em resumo

- PL: variáveis contínuas, tudo linear, ótimo em vértice, resolvida eficientemente pelo simplex.
- PI: algumas (PLIM) ou todas (PLI) as variáveis são inteiras; PB quando são todas binárias.
- Inteireza aparece por *natureza* (indivisibilidade) ou como *ferramenta de modelagem* (binárias para decisões e lógica).

Exercícios de fixação

1.1 Classifique como PL, PLIM, PLI ou PB: (a) escolher quais de 10 projetos financiar; (b) decidir quantas unidades inteiras produzir de cada brinquedo e se compra ou não cada molde; (c) escolher valores contínuos x_1,x_2 e uma binária y que diz qual de duas restrições vale.

1.2 Para cada grandeza, diga se uma variável contínua é uma boa aproximação ou se precisamos de inteiros ou binários: (a) toneladas de café transportadas; (b) número de caminhões; (c) abrir ou não um armazém; (d) litros de combustível.

1.3 No exemplo das vans, mude o preço da van para R$ 300. O ótimo inteiro muda? Arredondar a PL passa a dar o ótimo?

Capítulo 2

Por que não basta arredondar

A primeira ideia de quase todo mundo é: *"resolvo como PL, que é rápido, e arredondo o resultado"*. A promessa é sedutora: aproveitar a eficiência do simplex e a "semelhança" entre o problema inteiro e o contínuo. Este capítulo mostra, com três exemplos, por que essa ideia falha.

2.1 O exemplo de Wolsey

$$\max\ x_1 + 0{,}64\,x_2 \quad \text{s.a.} \quad 50x_1 + 31x_2 \le 250, \quad 3x_1 - 2x_2 \ge -4, \quad x_1, x_2 \ge 0 \text{ inteiros.}$$

Passo 1 – resolver a PL (ignorando a inteireza). O ótimo da PL está num vértice. O vértice "interessante" é o encontro das duas retas:

$$\begin{cases} 50x_1 + 31x_2 = 250 \\ 3x_1 - 2x_2 = -4 \end{cases} \Rightarrow\ x_2 = \tfrac{3x_1+4}{2} \ \Rightarrow\ 50x_1 + \tfrac{31(3x_1+4)}{2} = 250 \ \Rightarrow\ 193x_1 = 376.$$

Logo $x_1 = 376/193 \approx 1{,}95$ e $x_2 = 950/193 \approx 4{,}92$, com valor $984/193 \approx 5{,}098$. Os outros vértices, (0,0), (5,0) e (0,2), valem 0, 5 e 1,28. Então o ótimo da PL é (1,95; 4,92).

Passo 2 – arredondar. Os quatro vizinhos inteiros do ótimo da PL são:

ponto	$50x_1 + 31x_2 \le 250$?	$3x_1 - 2x_2 \ge -4$?	viável?	valor
(1,4)	174 ✓	-5 ×	não	–
(1,5)	205 ✓	-7 ×	não	–
(2,4)	224 ✓	-2 ✓	**sim**	4,56
(2,5)	255 ×	-4 ✓	não	–

O arredondamento "natural" (2,5) é **inviável**. O único vizinho viável vale 4,56.

Passo 3 – o ótimo inteiro de verdade. Testando todos os pontos inteiros viáveis (são poucos; veja a figura), o melhor é (5,0), com valor 5, **do outro lado** do polígono. O segundo melhor é (3,3), com 4,92.

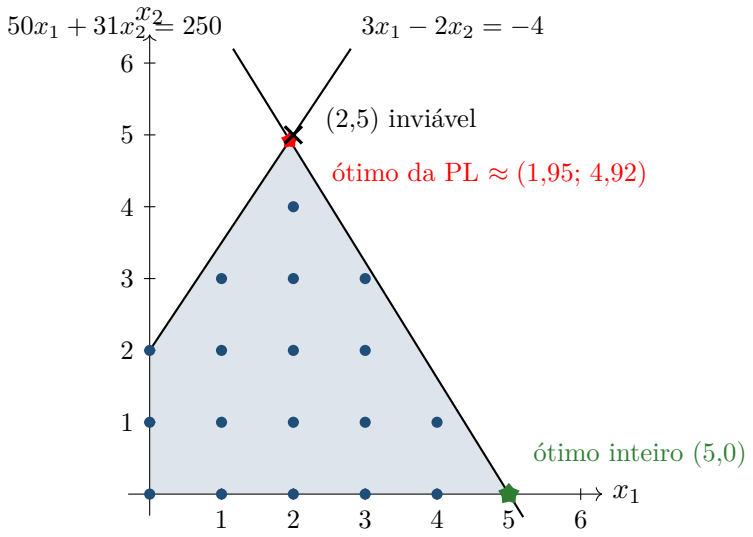


Figura 2.1: Exemplo de Wolsey: região viável da PL (azul claro), pontos inteiros viáveis (pontos azuis), ótimo da PL (estrela vermelha) e ótimo inteiro (estrela verde). O ótimo inteiro está longe do ótimo da PL.

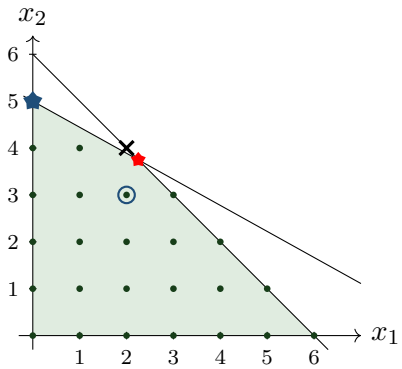
Lição

Arredondar a solução da PL pode dar (1) uma solução **inviável** ou (2) uma solução viável mas **longe do ótimo**. O ótimo inteiro pode estar "do outro lado" da região viável.

2.2 Outro exemplo clássico: o livro do MIT

Bradley, Hax e Magnanti [6, Seç. 9.4] usam um exemplo igualmente pequeno para mostrar o mesmo fenômeno:

$$\max\ 5x_1 + 8x_2 \quad \text{s.a.} \quad x_1 + x_2 \le 6, \quad 5x_1 + 9x_2 \le 45, \quad x_1, x_2 \ge 0 \text{ inteiros.}$$



	x_1	x_2	z
ótimo da PL (★)	2,25	3,75	41,25
arredondado (×)	2	4	inviável
viável mais próximo (○)	2	3	34
ótimo inteiro (★)	0	5	**40**

O ponto inteiro mais próximo do ótimo da PL é inviável ($5 \cdot 2 + 9 \cdot 4 = 46 > 45$), e o ponto viável mais próximo vale 34, longe do ótimo 40. Os autores observam ainda que, mudando a escala dos coeficientes, dá para construir exemplos em que o ótimo inteiro fica **tão longe quanto se queira** da solução arredondada, tanto em valor quanto em distância no plano.

2.3 O caso binário é ainda pior

$$\max\ 12x_1 + 7x_2 + 6x_3 \quad \text{s.a.} \quad 3x_1 + 2x_2 + 2x_3 \le 4, \quad x \in \{0,1\}^3.$$

Pense em três itens com valores 12, 7 e 6 e pesos 3, 2 e 2, numa mochila de capacidade 4. Uma estratégia "gulosa" razoável é pegar primeiro o item com maior **razão valor/peso**: $12/3 = 4$, $7/2 = 3{,}5$ e $6/2 = 3$. A gulosa pega o item 1 (peso 3) e não cabe mais nada: valor **12**. Mas olhe a enumeração completa:

x_1	x_2	x_3	peso	valor	viável?
0	0	0	0	0	sim
0	0	1	2	6	sim
0	1	0	2	7	sim
0	1	1	4	**13**	**sim**
1	0	0	3	12	sim
1	0	1	5	18	não
1	1	0	5	19	não
1	1	1	7	25	não

O ótimo é levar os itens 2 e 3 (valor 13). E "arredondar" uma variável binária fracionária significa jogá-la para 0 ou para 1, ou seja, tomar uma *decisão completamente diferente*. Com binárias não existe "perto".

2.4 Quando a inteireza é natural: escala de enfermeiras

Cada enfermeira trabalha **5 dias consecutivos** e folga 2. A demanda de enfermeiras nos dias de domingo a sábado é $d = (17{,}13{,}15{,}19{,}14{,}16{,}11)$. Queremos contratar o mínimo de enfermeiras.

A escolha da variável é o pulo do gato. Se usássemos "número de enfermeiras trabalhando no dia j", não conseguiríamos garantir os 5 dias consecutivos. A variável certa é

$$x_k = \text{número de enfermeiras que } \textbf{começam} \text{ a escala no dia } k, \qquad k = 0,\ldots,6 \text{ (dom,}\ldots\text{,sáb).}$$

Quem começa no dia k trabalha nos dias $k, k+1, \ldots, k+4$ (contando "em círculo": depois de sábado vem domingo). Então o dia j é coberto por quem começou em $j, j-1, j-2, j-3$ ou $j-4$. Por exemplo, o domingo ($j = 0$) é coberto por quem começou em dom, sáb, sex, qui e qua:

$$x_0 + x_3 + x_4 + x_5 + x_6 \geq 17.$$

O modelo completo é

$$\min \sum_{k=0}^{6} x_k \quad \text{s.a.} \quad \sum_{k:\,(j-k) \bmod 7 \leq 4} x_k \geq d_j \quad (j = 0,\ldots,6), \qquad x_k \in \mathbb{Z}_+.$$

A condição $(j-k) \bmod 7 \leq 4$ diz exatamente "o dia j está entre os 5 dias de quem começou em k". No notebook ela aparece como `(j - k) % 7 <= 4`.

Resultados.

	solução	total
PL (contínua)	(1,33; 5,33; 0; 7,33; 0; 3,33; 5)	22,33
PL arredondada para cima	(2,6,0,8,0,4,5)	25
Ótimo inteiro	(2,6,0,7,0,3,5)	**23**

"6,33 enfermeiras" não faz sentido. Arredondar *cada* variável para cima garante viabilidade (só aumentamos a cobertura), mas gasta 2 enfermeiras a mais que o necessário. O ótimo inteiro é 23 e, como a PL vale 22,33, sabemos que *nenhuma* escala usa 22 ou menos (a PL é um limite inferior; veja o Capítulo 3).

Cuidado: a solução da PL não é única

A PL das enfermeiras tem vários ótimos com valor 22,33. Outro solver pode devolver outra solução fracionária (e outro arredondamento). O valor 22,33, porém, é sempre o mesmo.

Exercícios de fixação

2.1 No exemplo de Wolsey, verifique que (3,3) é viável e calcule seu valor. Por que ele não aparece entre os "vizinhos" do ótimo da PL?

2.2 Na mochila de 3 itens, a capacidade passa a ser 5. Refaça a gulosa e a enumeração. A gulosa acerta agora?

2.3 Escreva a restrição de cobertura da quarta-feira ($j = 3$) no modelo das enfermeiras.

Capítulo 3

Relaxação linear, limitantes e qualidade de um modelo

Este capítulo apresenta o conceito que mais aparece no notebook, mesmo quando não é citado pelo nome: a **relaxação linear**. Ela explica por que o valor de M importa, por que uma formulação "com mais restrições" pode ser melhor e o que o solver faz por baixo dos panos.

3.1 Relaxação linear

Definição: relaxação linear

A **relaxação linear** de um problema inteiro é o mesmo problema sem as exigências de integralidade: $x \in \mathbb{Z}_+$ vira $x \geq 0$ e $x \in \{0,1\}$ vira $0 \leq x \leq 1$. Todo o resto (objetivo e restrições) fica igual.

Como a relaxação aceita *todas* as soluções do problema inteiro e mais algumas (as fracionárias), vale sempre:

Em **maximização**:	$z_{\text{PL}} \geq z_{\text{PI}}$	(a relaxação dá um **limite superior**);
Em **minimização**:	$z_{\text{PL}} \leq z_{\text{PI}}$	(a relaxação dá um **limite inferior**).

Nos exemplos do capítulo anterior: Wolsey (max) tem $5{,}098 \geq 5$; enfermeiras (min) tem $22{,}33 \leq 23$.

Duas consequências úteis:

- Se a solução ótima da relaxação **já é inteira**, ela é ótima para o problema inteiro.
- Se a relaxação é **inviável**, o problema inteiro também é.

No notebook, a função `valor_relaxado(prob)` calcula o valor da relaxação linear de um modelo (ela troca temporariamente todas as variáveis para contínuas, resolve e restaura; veja a Seção 5.4).

3.2 Limitantes e *gap*

Durante a resolução, o solver mantém dois números (pensando em maximização):

- a **incumbente**: a melhor solução inteira encontrada até agora, cujo valor é um **limite inferior** para o ótimo (o ótimo é pelo menos isso);

- um **limite superior**, que vem das relaxações (o ótimo é no máximo isso).

A diferença relativa entre eles é o **gap**:

$$\text{gap} = \frac{\text{limite superior} - \text{incumbente}}{\text{incumbente}}.$$

Quando o gap chega a zero, o solver **provou** a otimalidade.

Exemplo prático: mochila de 10 projetos do Bloco 6

A heurística gulosa encontra uma carteira de valor 129 (incumbente), e a relaxação linear diz que não dá para passar de 161,12 (limite superior). O gap inicial é $(161{,}12 - 129)/129 \approx 25\%$. O ótimo (142) está em algum lugar entre os dois. "Fechar" esse intervalo é o trabalho do *branch-and-bound*.

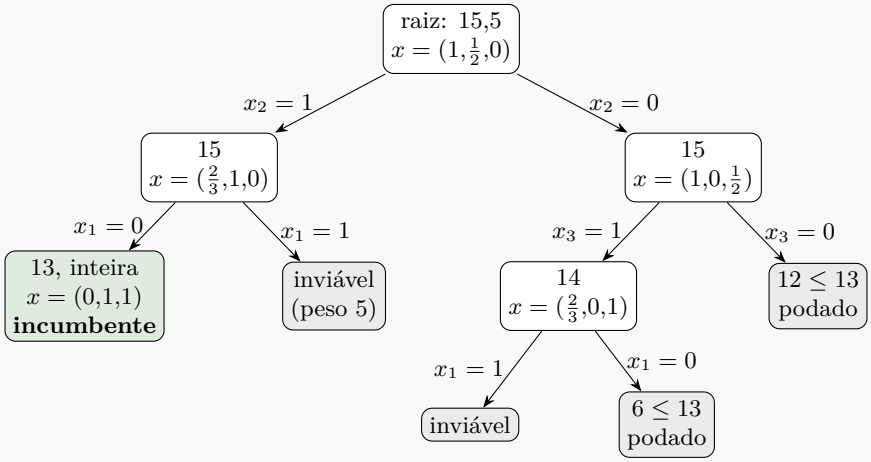
3.3 *Branch-and-bound* em uma página

Você não precisa implementar o *branch-and-bound* (B&B) neste notebook, porque o solver CBC faz isso. Mas entender a ideia explica **por que modelos com relaxação forte resolvem mais rápido**.

1. Resolva a relaxação linear. Se a solução é inteira, acabou.
2. Se alguma variável x_j tem valor fracionário v, **ramifique** (*branch*): crie dois subproblemas, um com $x_j \leq \lfloor v \rfloor$ e outro com $x_j \geq \lceil v \rceil$ (para binárias: $x_j = 0$ e $x_j = 1$). Toda solução inteira está em um dos dois.
3. Em cada subproblema, resolva a relaxação. **Pode** (*bound*) o nó se: (a) for inviável; (b) a relaxação for inteira, caso em que ela vira candidata a incumbente; ou (c) o limite do nó não for melhor que a incumbente.
4. Repita até não sobrar nó aberto.

Exemplo prático: B&B na mochila de 3 itens

$\max 12x_1 + 7x_2 + 6x_3$ s.a. $3x_1 + 2x_2 + 2x_3 \leq 4$, $x \in \{0,1\}^3$. A relaxação de uma mochila se resolve pela gulosa *fracionando* o último item que não cabe inteiro (Capítulo 12).



O ramo $x_2 = 1$ encontra a incumbente 13. O ramo $x_2 = 0$ tem limite $15 > 13$, então ainda precisa ser explorado; dentro dele, todos os nós acabam inviáveis ou com valor ≤ 13.

Conclusão: **13 é ótimo**. Se a relaxação fosse mais "apertada" (limites menores), mais nós seriam podados cedo.

Outros autores: de Land e Doig aos solvers atuais

- A ideia de limitar com relaxações lineares e dividir o problema vem de Land e Doig [7]. A ramificação em dois ramos, $x_j \leq \lfloor v \rfloor$ e $x_j \geq \lceil v \rceil$, descrita acima, é a variante de Dakin [10], adotada por praticamente todos os solvers.
- Bradley, Hax e Magnanti descrevem o B&B como uma estratégia de "dividir para conquistar": particionar a região viável em pedaços mais fáceis e, se preciso, particionar os pedaços de novo [6, Seç. 9.5].
- Os **planos de corte** de Gomory [9] acrescentam desigualdades que cortam a solução fracionária da relaxação sem cortar nenhum ponto inteiro. Os solvers modernos, como o CBC usado no notebook, combinam as duas ideias (*branch-and-cut*).
- O progresso é impressionante. Koch et al. mediram, de 2001 a 2020, um ganho de cerca de **50**× nos algoritmos de PLIM e **20**× no hardware: cerca de **1000**× no total, ou seja, o que levava 15 minutos passou a levar menos de 1 segundo [12]. Bixby faz um histórico dessa evolução desde os anos 1990 [11].

3.4 Formulações: o mesmo problema, modelos diferentes

Definição: poliedro e formulação

Um **poliedro** é um conjunto da forma $P = \{x \in \mathbb{R}^n : Ax \leq b\}$. Um poliedro P é uma **formulação** para um conjunto de pontos inteiros X se $X = P \cap \mathbb{Z}^n$, isto é, os pontos inteiros de P são exatamente as soluções do problema.

O mesmo conjunto X admite **infinitas formulações**: basta "inclinar" as restrições sem cortar nem acrescentar pontos inteiros. Todas dão o mesmo ótimo inteiro, mas **relaxações diferentes**.

Definição: formulação mais forte e formulação ideal

Se P_1 e P_2 são formulações para X e $P_1 \subset P_2$, dizemos que P_1 é **mais forte** (melhor) que P_2: sua relaxação é mais apertada e dá limites melhores. A formulação **ideal** é a **envoltória convexa** $\text{conv}(X)$, o menor poliedro convexo que contém todos os pontos de X. Nela, todos os vértices são inteiros e a relaxação linear já resolve o problema inteiro.

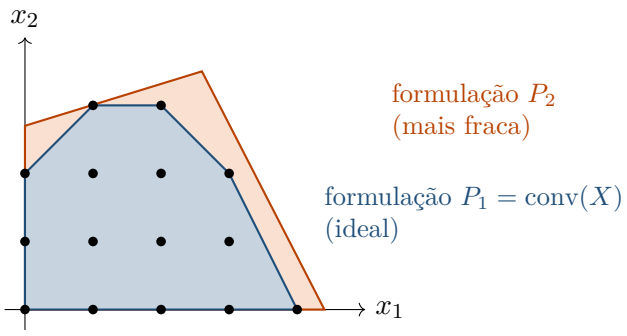


Figura 3.1: Duas formulações para o mesmo conjunto de pontos inteiros (pontos pretos). Ambas contêm exatamente os mesmos pontos inteiros, mas a relaxação de P_2 inclui muito mais pontos fracionários.

Outros autores: para ir além em formulações

O survey de Vielma [13] é a referência moderna sobre como obter formulações mais fortes (e menores), inclusive para disjunções como as do Bloco 2. As notas de aula do curso 15.083J do MIT (Bertsimas e Schulz) dedicam as primeiras aulas a formulações e formulações ideais [28].

Cuidado: "menos restrições" $\neq$ "modelo melhor"

Um modelo com mais restrições pode ser *mais forte*: sua relaxação é menor e o B&B poda mais. Você verá isso na prática em dois experimentos do notebook: o valor de M (Blocos 1 e 2) e a formulação agregada $\times$ desagregada (Bloco 3).

Exercícios de fixação

3.1 Em um problema de *minimização*, a relaxação deu 40 e a melhor solução inteira conhecida vale 50. O ótimo pode ser 38? Pode ser 45? Pode ser 52? Qual o gap?

3.2 A relaxação linear de um problema binário devolveu uma solução com todas as variáveis em 0 ou 1. O que você pode concluir?

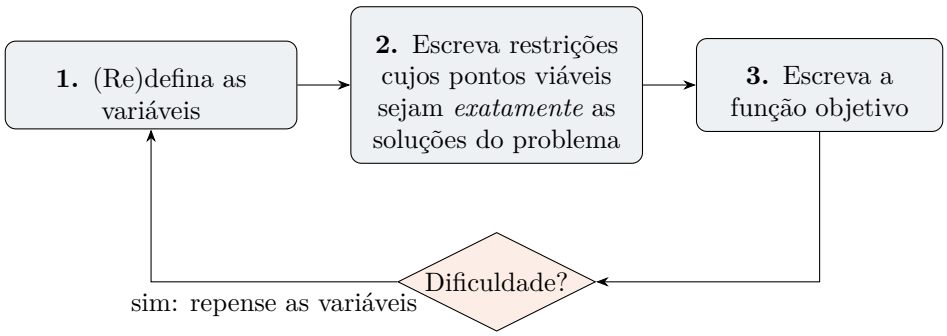
3.3 Por que, no exemplo do B&B, não foi necessário explorar o ramo $x_3 = 0$ além do primeiro nó?

Capítulo 4

Como formular modelos com variáveis inteiras

4.1 O "algoritmo" de formulação de Wolsey

Modelar não é adivinhar: há um roteiro que funciona para quase tudo.



- **Variáveis primeiro.** A escolha das variáveis define o que você consegue escrever. No exemplo das enfermeiras, "quantas *começam* no dia k" resolveu o problema dos 5 dias consecutivos.
- **Restrições: nem mais nem menos.** Cada solução do problema real tem que satisfazer todas as restrições (senão você corta soluções boas) e cada ponto que satisfaz as restrições tem que ser uma solução verdadeira (senão o solver "trapaceia").
- **Dificuldade?** Se algo não sai linear, quase sempre falta uma variável (em geral, binária).

Receita: perguntas para fazer antes de escrever qualquer código

1. O que eu *decido*? (variáveis e seus tipos: contínua, inteira, binária)
2. O que eu quero *otimizar*? (objetivo; cuidado com custos que só existem "se" algo acontece)
3. O que *limita* as decisões? (recursos, demandas, regras de negócio, lógica)
4. Existe alguma regra do tipo "se", "ou", "pelo menos", "no máximo", "só um"? Então precisaremos de binárias (Seções 4.3 e 4.5).

4.2 Variáveis binárias como indicadores de eventos

Uma variável binária representa uma **decisão sim/não** (fazer ou não fazer, comprar ou não comprar) ou **sinaliza um evento**:

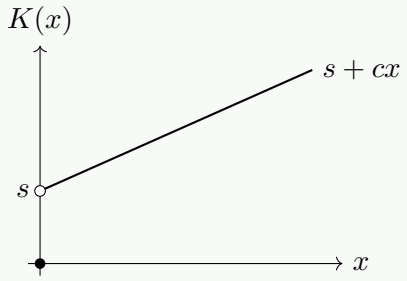
$$y = \begin{cases} 1 & \text{se o evento ocorre,} \\ 0 & \text{caso contrário.} \end{cases}$$

O desafio é sempre o mesmo: **amarrar** a binária às outras variáveis usando apenas restrições lineares. A ferramenta principal para isso é o *big-M*.

4.3 A técnica do *big-M*

Exemplo prático: custo fixo, a motivação

Produzir x unidades custa $K(x) = s+cx$ se $x > 0$ e $K(0) = 0$: há um custo fixo s (preparação de máquina, compra de molde, taxa inicial do táxi) que só é pago se produzirmos algo.



A função dá um "salto" em $x = 0$, então não é linear. Solução: criar $y \in \{0,1\}$ (1 se produzimos) e escrever

$$K = s\,y + c\,x, \qquad x \le M\,y,$$

onde M é a maior produção possível. Se $y = 0$, a restrição força $x \le 0$ (não produz, não paga). Se $y = 1$, ela vira $x \le M$, que não atrapalha (produz à vontade e paga s).

O raciocínio acima é um caso particular de um resultado geral.

Teorema 4.1 (Teorema das variáveis binárias, Sierksma)**.** *Seja $f : D \to \mathbb{R}$ uma função (do domínio D das variáveis x), $\delta \in \{0,1\}$ e $M \ge \max\{f(x) : x \in D\}$. Então, para todo $x \in D$,*

$$\big(\delta = 0 \;\Rightarrow\; f(x) \le 0\big) \quad \Longleftrightarrow \quad f(x) - M\delta \le 0.$$

Por que vale (por casos). Se $\delta = 0$, a restrição da direita é $f(x) \le 0$, exatamente o que a implicação exige. Se $\delta = 1$, a implicação não exige nada, e a restrição da direita vira $f(x) \le M$, que é sempre verdadeira porque M é pelo menos o maior valor de f. Nos dois casos, os dois lados dizem a mesma coisa. □

Trocando δ por $1 - \delta$ e f por $-f$ obtemos todas as variações de que precisamos (com $M \ge \max f$ e $m \le \min f$ no domínio):

Quero dizer…	Restrição linear
$\delta = 0 \Rightarrow f(x) \leq 0$	$f(x) \leq M\delta$
$\delta = 1 \Rightarrow f(x) \leq 0$	$f(x) \leq M(1-\delta)$
$\delta = 0 \Rightarrow f(x) \geq 0$	$f(x) \geq m\delta$
$\delta = 1 \Rightarrow f(x) \geq 0$	$f(x) \geq m(1-\delta)$
$x > 0 \Rightarrow y = 1$ (com $0 \leq x \leq M$)	$x \leq My$
$y = 1 \Rightarrow x \geq \ell$ (com $x \geq 0$)	$x \geq \ell\, y$

Exemplo prático: lendo uma restrição *big-M* pelos casos

Variáveis x_1,$x_2 \in [0{,}10]$ e a regra "se $\delta = 1$ então $x_1 + x_2 \leq 5$". Aqui $f = x_1 + x_2 - 5$, cujo maior valor é $10 + 10 - 5 = 15$. Logo

$$x_1 + x_2 - 5 \leq 15(1 - \delta).$$

Teste: $\delta = 1 \Rightarrow x_1 + x_2 \leq 5$ ✓. $\delta = 0 \Rightarrow x_1 + x_2 \leq 20$, que sempre vale no domínio, então a restrição fica "desligada" ✓.

4.4 Como escolher o valor de M

Receita: o menor M válido

M deve ser o **maior valor que f pode assumir** nas situações em que a restrição deve ficar desligada. Calcule-o com os limites das variáveis (ou resolvendo uma pequena PL, como no Bloco 2). Use o menor valor que *nunca corta* uma solução válida.

Cuidado: M pequeno demais × grande demais

- **Pequeno demais:** a restrição "desligada" ainda corta soluções válidas. O modelo fica **errado** e o ótimo pode piorar. Ex.: se a fábrica consegue produzir 150 bonecas mas você usa $M = 100$, proibiu produzir mais de 100.
- **Grande demais:** o modelo continua correto, mas (1) a relaxação linear fica **fraca**: com $M = 10^6$, $x \leq My$ aceita $y = x/10^6 \approx 0$, e o custo fixo quase some da relaxação; e (2) há **problemas numéricos**: o solver considera inteiro qualquer valor a menos de uma tolerância (por exemplo, 10^{-6}). Com $M = 10^6$ e $y = 10^{-6}$ "igual a zero", a restrição permite $x = 1$ sem pagar o custo fixo!

Outros autores: o que dizem os fabricantes de solvers

- A Gurobi chama o problema numérico acima de *trickle flow* ("fluxo que vaza"). No exemplo da documentação, com $y \leq 1\,000\,000\,x$ (x binária) e a tolerância de integralidade padrão de 10^{-5}, o valor $x = 0{,}0000099999$ é aceito como "zero" e permite $y = 9{,}9999$. A recomendação é usar o **menor M válido** e os limites mais apertados possíveis para as variáveis [18, 19].
- A documentação do AMPL recomenda que os limites usados no *big-M* sejam tão apertados quanto possível, idealmente da ordem de $\pm 10^4$, e nunca maiores que o inverso da tolerância de integralidade [20].

- Bradley, Hax e Magnanti já davam o mesmo conselho em 1977: em geral é melhor usar o menor valor possível de M para evitar dificuldades numéricas [6, Seç. 9.2].
- Solvers comerciais (Gurobi, CPLEX) oferecem **restrições indicadoras**, em que você escreve "$y = 1 \Rightarrow f(x) \leq 0$" diretamente e o solver cuida da lógica sem um M explícito. No notebook usamos PuLP/CBC, então escrevemos o *big-M* à mão, e entender o *big-M* é o que permite usar bem essas ferramentas.

4.5 Lógica com variáveis binárias

Uma **proposição** é uma afirmação que é verdadeira ou falsa ("o investimento 2 é escolhido", "a máquina opera no modo A"). Associamos a cada proposição simples P_i uma binária δ_i, com $\delta_i = 1$ se e somente se P_i é verdadeira. Proposições compostas usam conectivos:

conectivo	leitura	símbolo	tabela-verdade para (P_1,P_2) = (1,1)	(1,0)	(0,1)	(0,0)
negação	não P_1	$\neg P_1$	0	0	1	1
conjunção	P_1 e P_2	$P_1 \wedge P_2$	1	0	0	0
disjunção inclusiva	P_1 ou P_2 (ou ambos)	$P_1 \vee P_2$	1	1	1	0
disjunção exclusiva	exatamente um	$P_1 \triangledown P_2$	0	1	1	0
implicação	se P_1 então P_2	$P_1 \Rightarrow P_2$	1	0	1	1
equivalência	P_1 se e só se P_2	$P_1 \Leftrightarrow P_2$	1	0	0	1

Cuidado: a implicação só é falsa num caso

"Se chover, levo guarda-chuva" só é *mentira* se chover e eu *não* levar. Se não chover, levar ou não levar está tudo bem. Por isso $P_1 \Rightarrow P_2$ é falsa apenas em $(P_1,P_2) = (1,0)$.

Receita: transformar qualquer regra lógica em restrição: "proibir as linhas falsas"

1. Escreva a tabela-verdade e veja **em quais linhas a regra é falsa**.
2. Procure uma desigualdade linear nas δ_i que seja violada *exatamente* nessas linhas.
3. Atalho que sempre funciona para uma cláusula "ou": $L_1 \vee L_2 \vee \cdots$ vira $\sum(\text{literais}) \geq 1$, trocando cada literal negado $\neg P_i$ por $(1 - \delta_i)$.

Exemplo. $P_1 \Rightarrow P_2$ equivale a $\neg P_1 \vee P_2$, que vira $(1 - \delta_1) + \delta_2 \geq 1$, ou seja, $\delta_1 - \delta_2 \leq 0$. Conferindo: a única linha que viola $\delta_1 \leq \delta_2$ é (1,0), exatamente a linha falsa ✓.

Proposição	Restrição	Linha(s) proibida(s)
$\neg P_1$	$\delta_1 = 0$	$\delta_1 = 1$
$P_1 \wedge P_2$	$\delta_1 + \delta_2 = 2$	todas exceto (1,1)
$P_1 \vee P_2$	$\delta_1 + \delta_2 \geq 1$	(0,0)
$P_1 \triangledown P_2$	$\delta_1 + \delta_2 = 1$	(0,0) e (1,1)
$P_1 \Rightarrow P_2$	$\delta_1 - \delta_2 \leq 0$	(1,0)
$P_1 \Leftrightarrow P_2$	$\delta_1 - \delta_2 = 0$	(1,0) e (0,1)
$P_1 \wedge \cdots \wedge P_k \Rightarrow P_{k+1} \vee \cdots \vee P_n$	$\sum_{i \leq k} \delta_i - \sum_{i > k} \delta_i \leq k - 1$	$(1,\ldots,1,0,\ldots,0)$
pelo menos / exatamente / no máximo k de n	$\sum \delta_i \geq k$ / $= k$ / $\leq k$	–

Por que a fórmula geral funciona? A regra $P_1 \wedge \cdots \wedge P_k \Rightarrow P_{k+1} \vee \cdots \vee P_n$ só é falsa quando *todos* os k primeiros são verdadeiros e *todos* os demais são falsos. Nessa linha, o lado esquerdo vale $k-0=k$, e a restrição $\leq k-1$ é violada. Em qualquer outra linha, ou algum dos k primeiros é 0, ou algum dos demais é 1, e o lado esquerdo fica $\leq k-1$.

Outros autores: o catálogo de restrições lógicas do livro do MIT

Bradley, Hax e Magnanti [6, Seç. 9.2] organizam as restrições lógicas em "famílias" que você vai reconhecer no notebook (adaptamos a notação: $y_j = 1$ quando a restrição j deve valer):

- **Restrições alternativas** (pelo menos uma de $f_1 \leq b_1$, $f_2 \leq b_2$): $f_j(x) - B_j(1-y_j) \leq b_j$ e $y_1 + y_2 \geq 1$. Como basta uma, dá para economizar uma variável: $y_2 = 1 - y_1$. É exatamente a disjunção do Bloco 2.
- **Restrições condicionais**: "se $f_1(x) > b_1$ então $f_2(x) \leq b_2$". Ela só é violada quando $f_1 > b_1$ *e* $f_2 > b_2$, então equivale a "$f_1 \leq b_1$ **ou** $f_2 \leq b_2$": de novo uma disjunção.
- **Alternativas k de p**: pelo menos k das p restrições devem valer: $f_j(x) - B_j(1-y_j) \leq b_j$ para todo j, e $\sum_j y_j \geq k$.
- **Alternativas compostas**: a região viável é a união de vários "blocos" de restrições; usa-se *a mesma* binária para todas as restrições de um bloco.

Outros autores: indicador de contagem (Unicamp)

Nas notas de MC558 da Unicamp [17], aparece uma receita útil para *saber* se pelo menos k de n condições binárias x_i estão satisfeitas, com uma binária z que vale 1 exatamente nesse caso:

$$k - n(1-z) \;\leq\; \sum_{i=1}^{n} x_i \;\leq\; k - 1 + n\,z.$$

Com $z=1$, a da esquerda exige $\sum x_i \geq k$; com $z=0$, a da direita exige $\sum x_i \leq k-1$. É a mesma ideia de "ida e volta" do Bloco 5, aplicada a uma soma. Para aprofundar a modelagem lógica, os livros de H. P. Williams são referência clássica [15, 16].

Exercícios de fixação

4.1 Variável semicontínua. Uma máquina ou fica desligada ($x=0$) ou opera entre 10 e 50 unidades/h. Modele com uma binária y.

4.2 Com x_1,$x_2 \in [0,8]$, modele "se $\delta = 0$ então $x_1 - x_2 \geq 2$".

4.3 Escreva a restrição para "se P_1 então *não* P_2" e confira pela tabela-verdade.

4.4 Dadas três restrições $f_i(x) \leq 0$ ($i = 1,2,3$) com $f_i \leq M_i$ no domínio, modele "pelo menos duas das três devem valer".

4.5 Por que usar $M = 10^9$ "para garantir" é uma má ideia? Dê os dois motivos.

Capítulo 5

PuLP: do modelo no papel ao código

O notebook usa o **PuLP 2.9.0**, uma biblioteca Python de código aberto do projeto COIN-OR para *escrever* modelos [29], junto com o solver **CBC** (*COIN-OR Branch and Cut*), que já vem instalado com ela e faz o *branch-and-cut* (Capítulo 3). O PuLP só *descreve* o modelo: quem resolve é o solver, e trocar de solver (GLPK, HiGHS, Gurobi…) exige mudar apenas a chamada de `solve`. A primeira célula do notebook instala a versão certa (no Colab, pode ser preciso reiniciar a sessão uma vez).

5.1 Anatomia de um modelo

Vamos escrever o exemplo das vans e ônibus do Capítulo 1 em PuLP, linha por linha.

```
import pulp


# 1) o problema: nome e sentido (LpMinimize ou LpMaximize)
prob = pulp.LpProblem("excursao", pulp.LpMinimize)


# 2) as variáveis: nome, limites e tipo (cat)
v = pulp.LpVariable("vans",   lowBound=0, cat="Integer")
o = pulp.LpVariable("onibus", lowBound=0, cat="Integer")


# 3) a função objetivo: a PRIMEIRA expressão (sem <=, >=, ==) somada ao problema
prob += 250*v + 600*o, "custo"


# 4) as restrições: expressões com <=, >= ou ==, cada uma com um nome único
prob += 12*v + 30*o >= 50, "lugares"


# 5) resolver com o CBC (msg=0 silencia o log)
prob.solve(pulp.PULP_CBC_CMD(msg=0))

# 6) ler a solução

print(pulp.LpStatus[prob.status])          # 'Optimal'
print(v.varValue, o.varValue)              # 2.0 1.0
print(pulp.value(prob.objective))          # 1100.0
```

`LpProblem(nome, sentido)`
: cria o modelo vazio.

`LpVariable(nome, lowBound, upBound, cat)`
: cria uma variável. `cat` pode ser `"Continuous"` (padrão), `"Integer"` ou `"Binary"`; também existem as constantes `pulp.LpContinuous` e `pulp.LpInteger`. Sem `lowBound` a variável pode ser negativa!

`prob += expressão, "nome"`
: se a expressão não tem comparação, ela vira a **função objetivo**; se tem (`<=`, `>=`, `==`), vira uma **restrição**.

`pulp.lpSum(...)`
: soma eficiente de muitos termos (equivale a $\sum$).

`prob.status`
: 1 = ótimo (`Optimal`); outros: `Infeasible`, `Unbounded`, `Not Solved`.

`x.varValue`
: valor da variável na solução; `pulp.value(prob.objective)` é o valor ótimo.

5.2 Índices, somatórios e dicionários

Modelos reais têm muitas variáveis indexadas. O padrão do notebook é criar listas ou dicionários por *compreensão* e usar `lpSum` com geradores.

```
n = 3
lucro = [20, 15, 10]

# lista de variáveis x[0], x[1], x[2]
x = [pulp.LpVariable(f"x_{i}", lowBound=0, cat="Integer") for i in range(n)]

# dicionário indexado por pares (i, j)
y = {(i, j): pulp.LpVariable(f"y_{i}_{j}", cat="Binary")
     for i in range(n) for j in range(n)}


prob += pulp.lpSum(lucro[i]*x[i] for i in range(n)), "lucro"      # sum_i p_i x_i

for j in range(n):                                                # uma restrição por
     j
    prob += pulp.lpSum(y[i, j] for i in range(n)) == 1, f"um_por_coluna_{j}"
```

Receita: do papel para o código

$\sum_i p_i x_i$	`pulp.lpSum(p[i]*x[i] for i in range(n))`
"para todo j" ($\forall j$)	um laço `for j in ...:` com um `prob +=` dentro
x_{ij}	`x[i, j]` (dicionário com chave tupla)
$\sum_{k:\,\text{condição}} x_k$	`pulp.lpSum(x[k] for k in K if condição)`

5.3 Armadilhas comuns em PuLP

Cuidado: o que o solver *não* aceita

- **Desigualdades estritas e "diferente":** não existem `<`, `>` nem `!=` em PL/PI. Use `>= 1` no lugar de `> 0` para inteiros, e reescreva "$\neq$" com binárias.
- **Produtos de variáveis:** `x*y` com x e y variáveis *não é linear* (o PuLP dá erro). Linearize (Bloco 5 mostra como).
- **`if` sobre variáveis:** `if x.varValue > 0:` dentro do modelo não funciona, porque na hora de construir o modelo a variável ainda não tem valor. `if` só pode depender de *dados*. Lógica sobre variáveis é feita com binárias e restrições.
- **Nomes repetidos:** duas restrições com o mesmo nome geram erro. Use f-strings com o índice: `f"capacidade_{i}"`.
- **Esquecer o objetivo:** sem objetivo, o notebook avisa "Falta definir a função objetivo".
- **Esquecer restrições:** o modelo pode ficar ilimitado (`Unbounded`), como acontece no Bloco 2 se você rodar a verificação antes de completar o `TODO`.

5.4 As funções auxiliares do notebook

A terceira célula do notebook ("Funções auxiliares") define ferramentas usadas em todas as verificações. Você não precisa alterá-las, mas precisa saber o que fazem.

`resolver(prob, mostrar=True, variaveis=True)`
: resolve com o CBC, imprime o status, o valor do objetivo, o tempo e as variáveis não nulas, e **devolve o valor ótimo** (ou `None` se não for ótimo ou se o modelo estiver incompleto).

`rodar(construtor, *args, **kwargs)`
: chama a sua função construtora (por exemplo, `modelo_fabrica`), pega o `prob` que ela devolve (sozinho ou como primeiro item de uma tupla) e chama `resolver`. É por isso que todas as funções de exercício terminam com `return prob, ...`.

`relaxado(prob)` e `valor_relaxado(prob)`
: `relaxado` é um gerenciador de contexto (`with`) que transforma todas as variáveis em contínuas e as restaura no fim; `valor_relaxado` usa-o para devolver o valor da **relaxação linear**. Binárias continuam com limites [0,1].

`conferir(chave, obtido, dica="")`
: compara o valor obtido com o gabarito `ESPERADO[chave]` (com tolerância relativa de 10^{-4}) e imprime "correto", "não bate" (com a dica) ou "ainda sem solução".

Exemplo prático: o padrão de todos os exercícios

```
def modelo_exemplo(dados, opcao=False):
    prob = pulp.LpProblem("exemplo", pulp.LpMaximize)

    x = ...                      # variáveis

    prob += ...                  # objetivo

    prob += ...                  # restrições

    if opcao:                    # variações do exercício (regra extra, simetria
    ...)
        prob += ...

    return prob, x               # rodar() usa o primeiro item


obj = rodar(modelo_exemplo, dados, opcao=True)   # constrói, resolve e imprime

conferir("chave_do_gabarito", obj)               # compara com o gabarito
```

Exercício de fixação

5.1 Escreva em PuLP o problema de Wolsey (Seção 2.1) com um parâmetro `inteiro` que escolhe entre variáveis inteiras e contínuas. (É exatamente o primeiro exercício do notebook; a resposta está no Capítulo 6.)

Parte II

Resolvendo o notebook passo a passo

Capítulo 6

Bloco 0 – Por que programação inteira? (aquecimento)

Este bloco revisita os exemplos do Capítulo 2, agora no código. Há um único exercício para completar (o modelo de Wolsey); o resto são células para rodar e interpretar.

6.1 0.1 – Exercício: o modelo de Wolsey em PuLP

Perguntas da discussão

- *Qual a ideia por trás de "resolver o PL e arredondar"?* Aproveitar a eficiência do simplex, supondo que o ótimo inteiro esteja "perto" do ótimo contínuo.
- *O ótimo do PL será inteiro?* Não: é (1,95; 4,92). *O arredondamento será viável?* (2,5) não é. *Será ótimo?* O único vizinho viável, (2,4), vale 4,56, e o ótimo inteiro é 5 em (5,0). Veja a Seção 2.1.

O que o exercício pede. A função já cria o problema e as duas variáveis; o tipo das variáveis depende do parâmetro `inteiro`. Faltam três linhas: o objetivo e as duas restrições.

Passo a passo.

1. Objetivo max $x_1 + 0{,}64x_2$: como o problema foi criado com `LpMaximize`, basta somar a expressão.
2. Restrição $50x_1 + 31x_2 \leq 250$: expressão com `<=`.
3. Restrição $3x_1 - 2x_2 \geq -4$: expressão com `>=`. Não é preciso "passar para $\leq$": o PuLP aceita `>=` diretamente.

```python
def modelo_wolsey(inteiro=True):
    cat = pulp.LpInteger if inteiro else pulp.LpContinuous
    prob = pulp.LpProblem("wolsey", pulp.LpMaximize)
    x1 = pulp.LpVariable("x1", lowBound=0, cat=cat)
    x2 = pulp.LpVariable("x2", lowBound=0, cat=cat)

    prob += 1.00*x1 + 0.64*x2, "obj"        # função objetivo

    prob += 50*x1 + 31*x2 <= 250, "c1"      # restrição 1

    prob += 3*x1 - 2*x2 >= -4, "c2"         # restrição 2
    return prob, x1, x2
```

Verificação. A célula roda o modelo duas vezes, com `inteiro=False` (PL) e com `inteiro=True` (PI), usando a função `rodar`. Os valores esperados são 5,0984 e 5. A célula seguinte desenha a Figura 2.1 e lista os quatro vizinhos inteiros do ótimo da PL, como na tabela da Seção 2.1.

Cuidado

- Escrever o objetivo depois das restrições funciona, mas ele precisa ser a única expressão *sem* comparação somada ao problema.
- Se aparecer "Falta definir a função objetivo", a linha do objetivo não foi executada. Lembre-se de rodar a célula da função *antes* da célula de verificação.

6.2 0.1 (continuação) – O caso binário

A célula da mochila de 3 itens não tem `TODO`: ela enumera todas as $2^3 = 8$ soluções com `itertools.product([0, 1], repeat=3)`, que gera todas as tuplas (x_1,x_2,x_3) com valores 0 ou 1. Para cada uma, calcula peso e valor e marca se é viável. A saída mostra o que vimos na Seção 2.3: a gulosa (razões 4, 3,5, 3) pega o item 1 e fica em 12; o ótimo é 13, com os itens 2 e 3.

O que levar desta célula

Enumerar funciona para $n = 3$ (8 casos), mas 2^n explode muito rápido. Gulosas são rápidas, mas sem garantia. É por isso que precisamos de modelos e solvers.

6.3 0.2 – Escala de enfermeiras

Também não há `TODO`, mas vale a pena ler o código com cuidado, porque ele usa padrões que você vai repetir.

```python
d = [17, 13, 15, 19, 14, 16, 11]          # demanda de dom..sáb

def modelo_enfermeiras(d, inteiro=True):
    prob = pulp.LpProblem("enfermeiras", pulp.LpMinimize)
    cat = pulp.LpInteger if inteiro else pulp.LpContinuous
    x = [pulp.LpVariable(f"x_{k}", lowBound=0, cat=cat) for k in range(7)]
    prob += pulp.lpSum(x), "total"
    for j in range(7):
        prob += pulp.lpSum(x[k] for k in range(7) if (j - k) % 7 <= 4) >= d[j], f"
    dia_{j}"
    return prob, x
```

- `x[k]` = enfermeiras que **começam** no dia k (veja a Seção 2.4 para entender por que essa é a variável certa).
- `pulp.lpSum(x)` soma todas as variáveis da lista: o total contratado.
- O laço cria uma restrição por dia j. O gerador soma apenas os x_k cujo turno de 5 dias inclui o dia j, isto é, $(j-k) \bmod 7 \in \{0,1,2,3,4\}$. O operador `%` do Python devolve sempre um resto entre 0 e 6, mesmo quando $j-k$ é negativo, o que faz a "volta" de sábado para domingo funcionar sozinha.

Saída e leitura. PL = 22,33, PI = 23, PL arredondada para cima = 25. A mensagem central: **arredondar para cima garante viabilidade, mas não otimalidade**; e o limite da PL (22,33) prova que 23 é o melhor possível, já que o total é inteiro e precisa ser $\geq 22{,}33$.

6.4 O mapa do resto da aula

A partir do Bloco 1, a inteireza deixa de vir da natureza do problema e passa a ser uma **ferramenta de modelagem**. Cada bloco ensina uma "receita" de linearização, e todas estão reunidas no Capítulo 14:

Bloco	Técnica
1	Implicações *se–então* (custo fixo, produção condicionada)
2	Restrições disjuntivas (*big-M*, escalonamento)
3	Relações lógicas (tabela-verdade, carteira de investimentos)
4	Representação de valores discretos
5	"Ou exclusivo" e indicadores (teoria de variáveis lógicas)
6	Problema da mochila binária
7	*Bin packing*

Capítulo 7

Bloco 1 – Implicações *se–então*

A ferramenta deste bloco é a **restrição de ativação**:

$$x > 0 \;\Rightarrow\; y = 1 \qquad \Longleftrightarrow \qquad x \le M\,y \qquad (M \ge \text{maior valor possível de } x).$$

Ela é o Teorema 4.1 com $f(x) = x$: "se $y = 0$ então $x \le 0$".

7.1 1A – Custo fixo: fabricar exige pagar a preparação

O problema. Uma fábrica produz Carrinho (A), Boneca (B) e Quebra-cabeça (C). Cada produto exige um **molde**, cujo custo fixo é pago uma única vez *se* o produto for fabricado, e zero se não for. Queremos maximizar o lucro líquido.

	Carrinho (A)	Boneca (B)	Quebra-cabeça (C)	disponível
lucro por unidade p_i (R$)	20	15	10	
horas de máquina h_i	3	1	2	$H = 200$
plástico q_i (kg)	1	1	2	$P = 150$
custo fixo do molde s_i (R$)	100	400	100	

Perguntas da discussão

1. *Quais são as variáveis? Só x_i basta?* Não. O custo fixo é "tudo ou nada": vale s_i para qualquer $x_i > 0$ e 0 para $x_i = 0$, o que não é linear em x_i (Seção 4.3). Precisamos de $x_i \in \mathbb{Z}_+$ (unidades) e $y_i \in \{0,1\}$ (comprar ou não o molde i).
2. *Como expressar "se $x_i > 0$ então $y_i = 1$"?* Com $x_i \le M_i y_i$.
3. *O que impede $y_i = 0$ com $x_i > 0$?* A própria restrição: $y_i = 0$ força $x_i \le 0$. *E nada impede $y_i = 1$ com $x_i = 0$, por que não é problema?* Porque $y_i = 1$ custa $s_i > 0$ no objetivo. Pagar um molde sem produzir só diminui o lucro, então o solver, maximizando, nunca escolhe isso.
4. *Que valor dar a M_i?* O maior x_i que os recursos permitem (detalhes abaixo).

Construindo o modelo, passo a passo

Passo 1 – variáveis. $x_i \in \mathbb{Z}_+$ = unidades do produto i; $y_i \in \{0,1\} = 1$ se o molde de i é comprado.

Passo 2 – objetivo. Lucro das unidades menos custos fixos *pagos*:

$$\max \sum_i \left(p_i x_i - s_i y_i\right) = 20x_A + 15x_B + 10x_C - 100y_A - 400y_B - 100y_C.$$

Passo 3 – recursos.

$$3x_A + x_B + 2x_C \le 200 \quad \text{(horas)}, \qquad x_A + x_B + 2x_C \le 150 \quad \text{(plástico)}.$$

Passo 4 – ativação. $x_i \le M_i y_i$ para $i = A,B,C$.

Passo 5 – escolher M_i. Mesmo usando *todo* o recurso só com o produto i, não dá para produzir mais que H/h_i (horas) nem mais que P/q_i (plástico). Logo $M_i = \min\{H/h_i,\, P/q_i\}$:

	H/h_i	P/q_i	M_i
A	$200/3 \approx 66{,}7$	150	66,7
B	200	150	150
C	100	75	75

Refinamento opcional: como x_i é inteiro, poderíamos usar $\lfloor 66{,}7 \rfloor = 66$ para A, um M ainda mais apertado.

O código

O esqueleto já cria variáveis e restrições de recursos. Faltam o objetivo (TODO 1) e a ativação dentro do laço (TODO 2), em que `Mi` já está calculado.

```
def modelo_fabrica(M=None, com_regra=False):
    n = len(lucro)
    prob = pulp.LpProblem("fabrica", pulp.LpMaximize)
    x = [pulp.LpVariable(f"x_{i}", lowBound=0, cat="Integer") for i in range(n)]
    y = [pulp.LpVariable(f"y_{i}", cat="Binary") for i in range(n)]

    prob += pulp.lpSum(horas[i]*x[i]    for i in range(n)) <= H, "horas"
    prob += pulp.lpSum(plastico[i]*x[i] for i in range(n)) <= P, "plastico"


    # TODO 1: lucro das unidades - custo fixo dos moldes comprados
    prob += pulp.lpSum(lucro[i]*x[i] - custo_fixo[i]*y[i] for i in range(n)), "
    lucro_liquido"


    # TODO 2: ativação x_i <= M_i * y_i
    for i in range(n):
        Mi = M if M is not None else min(H/horas[i], P/plastico[i])
        prob += x[i] <= Mi*y[i], f"ativa_{i}"


    if com_regra:                       # exercício 1B (próxima seção)
        prob += x[2] >= 20*y[1], "regra_B_implica_C"
    return prob, x, y
```

O parâmetro `M` existe para o experimento seguinte: com `M=None` usamos o M_i apertado de cada produto; com um número, o mesmo M para todos.

Resultado e interpretação

O ótimo é **1875**, com $x = (25{,}125{,}0)$ e $y = (1{,}1{,}0)$: produz 25 carrinhos e 125 bonecas e **não** produz quebra-cabeças. Conferindo: horas $3 \cdot 25 + 125 = 200$ e plástico $25 + 125 = 150$, os dois recursos esgotados. Lucro $500 + 1875 - 100 - 400 = 1875$.

Por que o quebra-cabeça fica de fora? Ele usa 2 kg de plástico por R$ 10 de lucro (R$ 5/kg), enquanto a boneca dá R$ 15/kg. Com o plástico como gargalo, não compensa pagar R$ 100 de molde para produzir C. Fixando cada combinação de moldes e otimizando a produção, obtemos:

y_A	y_B	y_C	lucro ótimo	x_A	x_B	x_C
1	1	0	**1875**	25	125	0
0	1	0	1850	0	150	0
1	1	1	1775	25	125	0
0	1	1	1750	0	150	0
1	0	0	1220	66	0	0
1	0	1	1130	66	0	1
0	0	1	650	0	0	75

Repare nas linhas com $y_C = 1$ e $x_C = 0$: o modelo *permite* comprar o molde sem usar, mas isso só tira R$ 100 do lucro. Por isso o solver nunca escolhe essa opção.

Cuidado: e a restrição $y_i \le x_i$?

Em sala apareceu também a restrição $y_A \le x_A$ ("se não produz, não compra o molde"). Ela é válida, porque x_A é inteiro e $y_A = 1$ exige $x_A \ge 1$, mas aqui é **desnecessária**: o custo fixo no objetivo já empurra y_i para 0 quando $x_i = 0$. Ela só seria necessária se $y_i = 1$ fosse "vantajoso" (por exemplo, um bônus por abrir) ou se outra restrição dependesse de y_i.

7.2 Experimento: o valor de M importa?

Todo M **válido** dá o mesmo ótimo inteiro (1875), mas a **relaxação linear** muda muito:

M	relaxação linear	gap até o ótimo inteiro
10^6	2374,95	499,95
1000	2322,50	447,50
200	2112,50	237,50
apertado (M_i)	2004,17	129,17

Por que isso acontece? Na relaxação, y_i pode ser fracionário. Para pagar o mínimo de custo fixo, o solver escolhe o menor y_i permitido: $y_i = x_i / M_i$. Com M apertado, a relaxação produz $x = (25{,}125{,}0)$ com $y_A = 25/66{,}7 = 0{,}375$ e $y_B = 125/150 \approx 0{,}833$; o custo fixo pago é $100 \cdot 0{,}375 + 400 \cdot 0{,}833 \approx 371$, e o valor fica $2375 - 371 \approx 2004$. Com $M = 10^6$, y_B fica em 0,000125: o custo fixo praticamente some e a relaxação vira o problema *sem* custo fixo (lucro 2375). Quanto maior o M, pior o limite e mais trabalho para o *branch-and-bound*. Além disso, M enorme gera problemas numéricos (Seção 4.4).

7.3 1B – Produção condicionada

A regra. *Se a fábrica produzir Boneca (B), é obrigada a produzir Quebra-cabeça (C) em lote de pelo menos 20 unidades.*

Perguntas da discussão

- *Como forçar $x_C \geq 20$ apenas quando $y_B = 1$?* Com $x_C \geq 20\, y_B$. Se $y_B = 1$: $x_C \geq 20$. Se $y_B = 0$: $x_C \geq 0$, que já valia.
- *Qual a contrapositiva? Precisa de outra restrição?* A contrapositiva é "se $x_C < 20$, então não produz B". Ela já está garantida: se $x_C < 20$, a restrição força $y_B = 0$, e a ativação $x_B \leq M_B y_B$ força $x_B = 0$. Uma única restrição basta.
- *A regra muda o ótimo?* Sim, porque o ótimo de 1A tinha $x_B = 125$ e $x_C = 0$, que agora é proibido.

Código. Dentro de `if com_regra:` (já mostrado acima):

```
    if com_regra:

        prob += x[2] >= 20*y[1], "regra_B_implica_C"      # x_C >= 20 * y_B
```

Lembre-se dos índices: `0`=Carrinho, `1`=Boneca, `2`=Quebra-cabeça.

Resultado. Ótimo **1375** com $x = (25{,}85{,}20)$ e $y = (1{,}1{,}1)$. Agora é preciso comprar o molde de C (R$ 100), e cada quebra-cabeça usa recursos que iriam para bonecas: horas $75+85+40 = 200$, plástico $25+85+40 = 150$. Lucro: $500+1275+200-600 = 1375$.

Cuidado: erros típicos do Bloco 1

- Esquecer $-s_i y_i$ no objetivo: o modelo "esquece" o custo fixo e devolve 2375.
- Usar M pequeno demais (por exemplo, $M = 50$ para todos): corta soluções válidas e o ótimo cai.
- Escrever $x_C \geq 20$ sem multiplicar por y_B: força produzir C *sempre*, e não apenas quando há boneca.
- Trocar índices (`y[2]` no lugar de `y[1]`): a regra passa a ser "se C então C".

Capítulo 8

Bloco 2 – Restrições disjuntivas

Às vezes queremos que **apenas uma** entre duas restrições valha ("ou isto, ou aquilo"). O conjunto viável deixa de ser convexo, e nenhuma PL pura consegue representá-lo. Com uma binária y:

Receita: disjunção "$f(x) \le 0$ OU $g(x) \le 0$"

$$f(x) \le M_1(1-y), \qquad g(x) \le M_2\, y, \qquad y \in \{0,1\}.$$

- $y = 1$: a primeira vira $f(x) \le 0$ (ativa) e a segunda vira $g(x) \le M_2$ (desligada).
- $y = 0$: a primeira vira $f(x) \le M_1$ (desligada) e a segunda vira $g(x) \le 0$ (ativa).

M_1 precisa ser grande o suficiente para não cortar nada quando g está ativa, e vice-versa.

8.1 2A – Modos de operação

O problema. Uma linha de produção opera no **modo A** (limite de energia: $4x_1 + 2x_2 \le 80$) **ou** no **modo B** (limite de insumo: $2x_1 + 5x_2 \le 100$); não precisa respeitar os dois ao mesmo tempo. O contrato exige $x_1 \ge 10$. Maximizar $2x_1 + 3x_2$.

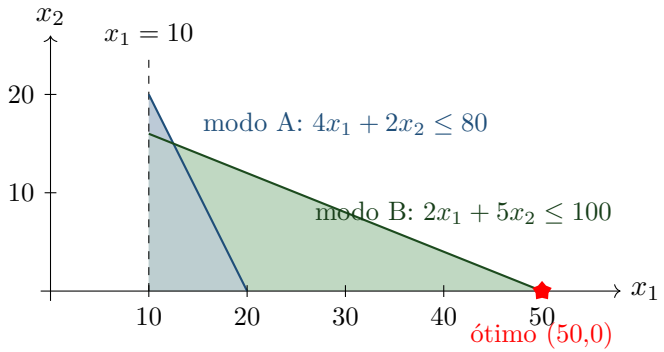


Figura 8.1: Região viável real do Bloco 2A: a **união** do triângulo do modo A (azul) com o triângulo do modo B (verde). A união não é convexa. O ótimo, com valor 100, está no modo B.

Perguntas da discussão

1. *A região é convexa? Onde fica o ótimo?* Não é convexa (é a união de dois triângulos). Avaliando os vértices: no modo A, (10,20) vale 80 e (20,0) vale 40; no modo B, (50,0) vale 100 e (10,16) vale 68. O ótimo é (50,0), com valor 100.

2. *Para que valor de y cada restrição está ativa?* Com $f(x) = 4x_1 + 2x_2 - 80$ e $g(x) = 2x_1 + 5x_2 - 100$: $y = 1$ ativa o modo A e $y = 0$ ativa o modo B.
3. *Qual o menor M_1 que nunca corta uma solução válida?* Quando $y = 0$ (modo B), a restrição $f(x) \leq M_1$ precisa aceitar todo ponto do modo B. Então $M_1 \geq \max\{f(x) : x \in D,\ g(x) \leq 0\}$, onde $D = \{x_1 \geq 10,\ x_2 \geq 0\}$. Isso é uma PL!

Exercício 2A-1: calcular M_1 e M_2 resolvendo duas PLs

$$M_1 = \max\{\, 4x_1 + 2x_2 - 80 \ :\ 2x_1 + 5x_2 \leq 100,\ x_1 \geq 10,\ x_2 \geq 0 \,\},$$

$$M_2 = \max\{\, 2x_1 + 5x_2 - 100 \ :\ 4x_1 + 2x_2 \leq 80,\ x_1 \geq 10,\ x_2 \geq 0 \,\}.$$

À mão, pelos vértices: para M_1, os vértices do triângulo B são (10,0), (50,0) e (10,16), com $f = -40,\ 120,\ -8$, logo $M_1 = 120$. Para M_2, os vértices do triângulo A são (10,0), (20,0) e (10,20), com $g = -80,\ -60,\ 20$, logo $M_2 = 20$.

No código, a função recebe as *funções* `obj` e `outra` (definidas com `lambda`) e as aplica às variáveis do PuLP. Isso funciona porque `4*x1 + 2*x2 - 80` com `x1, x2` sendo variáveis do PuLP gera uma expressão linear.

```
f = lambda x1, x2: 4*x1 + 2*x2 - 80     # modo A:  f(x) <= 0

g = lambda x1, x2: 2*x1 + 5*x2 - 100    # modo B:  g(x) <= 0

def maximizar_sobre(obj, outra):

    """max obj(x)  s.a.  x em D = {x1 >= 10, x2 >= 0}  e  outra(x) <= 0."""
    prob = pulp.LpProblem("M", pulp.LpMaximize)

    x1 = pulp.LpVariable("x1", lowBound=10)    # o domínio D já está nos limites
    x2 = pulp.LpVariable("x2", lowBound=0)

    prob += obj(x1, x2), "obj"                 # TODO: objetivo

    prob += outra(x1, x2) <= 0, "outra_ativa"  # TODO: a outra restrição está ativa
    return resolver(prob, mostrar=False)

M1 = maximizar_sobre(f, g)     # 120.0
M2 = maximizar_sobre(g, f)     # 20.0
```

Cuidado: o domínio importa

M_2 só é tão pequeno (20) porque usamos $x_1 \geq 10$. Sem essa informação, o ponto (0,40) seria do modo A e daria $g = 100$. Quanto mais você souber sobre as variáveis, menor (e melhor) o M.

Exercício 2A-2: o modelo disjuntivo

```
def modelo_disjuntivo(M1, M2):
    prob = pulp.LpProblem("disjuntivo", pulp.LpMaximize)
    x1 = pulp.LpVariable("x1", lowBound=10)
    x2 = pulp.LpVariable("x2", lowBound=0)

    y  = pulp.LpVariable("y", cat="Binary")      # y = 1 -> modo A ; y = 0 -> modo B
    prob += 2*x1 + 3*x2, "obj"
    prob += f(x1, x2) <= M1*(1 - y), "modoA"     # TODO
    prob += g(x1, x2) <= M2*y,       "modoB"     # TODO
    return prob, x1, x2, y
```

Resultado: **100**, em $x = (50{,}0)$ com $y = 0$ (modo B). Antes de completar o TODO, a verificação mostra "Unbounded": sem restrições além de $x_1 \geq 10$ e $x_2 \geq 0$, o objetivo cresce sem limite.

Cuidado: a chamada `rodar(modelo_disjuntivo, M1 or 120, M2 or 20)`

`M1 or 120` significa "use `M1` se ele existir; se for `None` (exercício anterior incompleto), use 120". Assim a verificação funciona mesmo que o 2A-1 não esteja pronto.

Leitura do gráfico: o que M faz com a relaxação

Eliminando y das duas restrições ($y \geq g/M_2$ e $y \leq 1 - f/M_1$), a região da relaxação linear é

$$\frac{f(x)}{M_1} + \frac{g(x)}{M_2} \leq 1, \qquad f(x) \leq M_1, \qquad g(x) \leq M_2.$$

Quanto maiores os M, mais "frouxa" essa desigualdade e maior a região laranja do gráfico do notebook:

(M_1,M_2)	limite da relaxação	ótimo inteiro
(120,20)	100	100
(120,100)	116	100
(500,500)	285,7	100
(5000,5000)	2214,3	100

Com os M mínimos, a relaxação já dá o ótimo exato (100). Com $M = 5000$, o limite passa de 2200. E o que acontece com M **menor** que o necessário (o *slider* do notebook)? Com $M_1 = 50$, por exemplo, no modo B a restrição "desligada" vira $4x_1 + 2x_2 \leq 130$, que corta o ponto (50,0). O "ótimo" cai para 82,5: **modelo errado**.

8.2 2B – Escalonamento em uma máquina

O problema. Seis tarefas, com tempo de processamento p_j e peso w_j, numa única máquina. Duas tarefas nunca são processadas ao mesmo tempo. Queremos minimizar a soma ponderada dos tempos de término, $\sum_j w_j C_j$.

tarefa	J1	J2	J3	J4	J5	J6
p_j	4	3	6	2	5	3
w_j	2	1	3	1	2	1
p_j/w_j	2	3	2	2	2,5	3

Perguntas da discussão

1. *Variáveis?* Início $s_j \geq 0$ (contínua) e, para cada par $j < k$, uma binária de ordem: $y_{jk} = 1$ se j vem antes de k, e 0 se k vem antes de j.
2. *Onde está a disjunção?* Para cada par: **ou** j termina antes de k começar ($s_j + p_j \leq s_k$), **ou** k termina antes de j começar ($s_k + p_k \leq s_j$).
3. *Que M vale?* Sem tempo ocioso desnecessário, tudo termina até $\sum_j p_j = 23$. Então $s_j + p_j - s_k \leq 23$ sempre, e $M = \sum_j p_j$ basta.
4. *Existe regra simples?* Sim: a **regra de Smith** (WSPT). Sequenciar por p_j/w_j crescente é ótimo quando não há datas de liberação.

O modelo

$$\begin{aligned} \min \quad & \sum_j w_j \, (s_j + p_j) \\ \text{s.a.} \quad & s_j + p_j \leq s_k + M(1 - y_{jk}) \quad \forall\, j < k \quad (\text{se } y_{jk} = 1\text{: } j \text{ antes de } k) \\ & s_k + p_k \leq s_j + M\, y_{jk} \qquad \forall\, j < k \quad (\text{se } y_{jk} = 0\text{: } k \text{ antes de } j) \\ & s_j \geq 0, \quad y_{jk} \in \{0{,}1\}. \end{aligned}$$

Como $C_j = s_j + p_j$, o objetivo é $\sum_j w_j C_j$. A disjunção vale para **cada par** $j < k$: são $\binom{6}{2} = 15$ binárias e 30 restrições.

```
def modelo_1maq(p, w, r=None):
    n = len(p)
    M = sum(p) + (max(r) if r else 0)
    prob = pulp.LpProblem("maquina_unica", pulp.LpMinimize)
    s = [pulp.LpVariable(f"s_{j}", lowBound=0) for j in range(n)]
    y = {(j, k): pulp.LpVariable(f"y_{j}_{k}", cat="Binary")

         for j in range(n) for k in range(j+1, n)}           # y=1: j antes de k

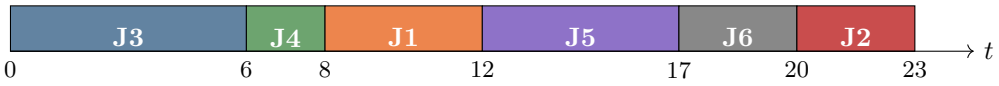

    # TODO 1: objetivo  sum_j w_j * C_j,  C_j = s_j + p_j
    prob += pulp.lpSum(w[j]*(s[j] + p[j]) for j in range(n)), "soma_ponderada"

    # TODO 2: as duas restrições disjuntivas para cada par j < k
    for (j, k), yjk in y.items():
        prob += s[j] + p[j] <= s[k] + M*(1 - yjk), f"j{j}_antes_k{k}"
        prob += s[k] + p[k] <= s[j] + M*yjk,       f"k{k}_antes_j{j}"

    # TODO 3 (desafio): datas de liberação
    if r:
        for j in range(n):
            prob += s[j] >= r[j], f"release_{j}"
    return prob, s, y
```

Lendo o laço. `y.items()` percorre os pares `((j, k), variável)`. Para cada par escrevemos as duas restrições: com $y_{jk} = 1$ a primeira está ativa e a segunda fica desligada (folga M); com $y_{jk} = 0$, o contrário.

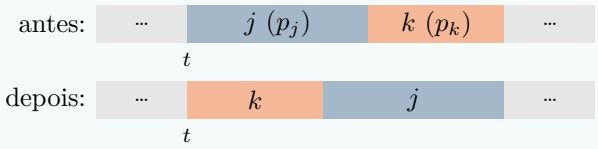
Resultado. Ótimo **127**. Uma sequência ótima devolvida pelo solver:



Conferindo: $3 \cdot 6 + 1 \cdot 8 + 2 \cdot 12 + 2 \cdot 17 + 1 \cdot 20 + 1 \cdot 23 = 18 + 8 + 24 + 34 + 20 + 23 = 127$. A regra de Smith (ordem J1, J3, J4, J5, J2, J6) também dá 127. Há empates nas razões, logo várias sequências ótimas.

Outros autores: por que a regra de Smith funciona: o argumento da troca

A regra foi publicada por W. E. Smith em 1956 [21]. A prova clássica é por **troca de vizinhos**. Suponha uma sequência em que j vem imediatamente antes de k, começando no instante t, e compare com a sequência que só troca os dois:



o resto da sequência não muda

As outras tarefas terminam nos mesmos instantes. Só mudam as contribuições de j e k:

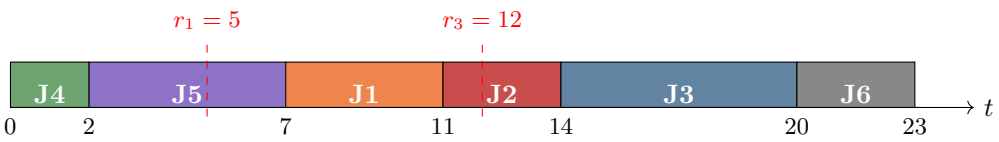
$$\Big(\underbrace{w_j(t+p_j) + w_k(t+p_j+p_k)}_{j \text{ antes de } k}\Big) - \Big(\underbrace{w_k(t+p_k) + w_j(t+p_k+p_j)}_{k \text{ antes de } j}\Big) = w_k p_j - w_j p_k.$$

Colocar j antes de k não é pior exatamente quando $w_k p_j \le w_j p_k$, isto é, $\dfrac{p_j}{w_j} \le \dfrac{p_k}{w_k}$. Numa sequência ótima, então, as razões aparecem em ordem crescente. O argumento *falha* com datas de liberação: trocar j e k pode obrigar a máquina a esperar.

Desafio: datas de liberação

Agora a tarefa j só pode começar em $s_j \ge r_j$, com $r = (5,0,12,0,0,0)$. Basta o TODO 3 acima, mas o M também precisa crescer: a máquina pode ficar ociosa esperando uma liberação, então usamos $M = \sum p_j + \max r_j$.

Resultado. Ótimo **135** (confirmado pela força bruta sobre as $6! = 720$ permutações). Uma sequência ótima:



A regra de Smith deixa de ser ótima: ela quer começar por J1 ou J3, que ainda não foram liberadas. (O valor 127 que o notebook imprime para "Smith ignorando r" é o de uma sequência que *viola* as liberações.)

Cuidado: erros típicos do Bloco 2

- Trocar y e $1 - y$ entre as duas restrições do mesmo par. Às vezes o modelo ainda funciona por simetria, mas a interpretação de y fica invertida, e no 2A o modo errado é ligado.
- Usar M que não cobre o maior r_j no desafio: corta sequências válidas.
- Escrever a disjunção só para pares consecutivos $(j,j+1)$: ela precisa valer para **todo** par $j < k$.
- No 2A-1, esquecer que $x_1 \geq 10$ faz parte do domínio: o M calculado fica maior que o necessário.

Conexão com a lista

A ideia de ordenar pares de tarefas com binárias e *big-M* é exatamente a das restrições de ordenação do *flowshop* permutacional (exercício 4 da lista de prática).

Capítulo 9

Bloco 3 – Relações lógicas

Associamos a cada proposição simples P_i uma binária δ_i ($\delta_i = 1 \iff P_i$ verdadeira). Proposições compostas viram **restrições lineares sobre os** δ_i. Releia a Seção 4.5 antes de começar: lá está a técnica de "proibir as linhas falsas" da tabela-verdade.

9.1 3A – Laboratório da tabela-verdade

Como o verificador funciona. Cada desafio é uma tupla `(nome, nº de variáveis, regra lógica, restrição linear)`. A função `verificar_logicas` testa **todas** as 2^n atribuições de 0/1 e compara: a restrição deve ser satisfeita *exatamente* nas linhas em que a regra é verdadeira. Você escreve a restrição como uma `lambda` que devolve `True`/`False`; para combinar várias restrições, use `and`.

Perguntas da discussão

- $\delta_1+\delta_2 \geq 1$ *representa qual conectivo?* O "ou inclusivo": só a linha (0,0) viola. *E* $\delta_1-\delta_2 \leq 0$*?* A implicação $P_1 \Rightarrow P_2$: só a linha (1,0) viola.
- *Como representar* $P_1 \Rightarrow P_2$*?* Ela só é falsa em $P_1 = 1, P_2 = 0$. Precisamos de uma restrição violada só nessa linha: $\delta_1 \leq \delta_2$.

Derivando cada linha do exercício

Regra	Raciocínio (linhas falsas → restrição)	`lambda`
$\neg P_1$	falsa em $\delta_1 = 1$	`a == 0`
$P_1 \wedge P_2$	verdadeira só em (1,1): a soma tem que ser 2	`a + b == 2`
$P_1 \vee P_2$	falsa só em (0,0): soma ≥ 1	`a + b >= 1`
$P_1 \triangledown P_2$	falsa em (0,0) e (1,1): soma exatamente 1	`a + b == 1`
$P_1 \Rightarrow P_2$	$\neg P_1 \vee P_2$: $(1-a)+b \geq 1$	`a - b <= 0`
$P_1 \Leftrightarrow P_2$	falsa em (1,0) e (0,1): iguais	`a - b == 0`
$P_1 \wedge P_2 \Rightarrow P_3$	falsa só em (1,1,0); fórmula geral com $k = 2$	`a + b - c <= 1`
$P_1 \Rightarrow P_2 \vee P_3$	falsa só em (1,0,0); fórmula geral com $k = 1$	`a - b - c <= 0`
pelo menos 2 de 4	contar os verdadeiros	`a + b + c + d >= 2`
exatamente 2 de 4	idem	`a + b + c + d == 2`
no máximo 1 de 4	exclusão mútua	`a + b + c + d <= 1`

Exemplo prático: conferindo $P_1 \wedge P_2 \Rightarrow P_3$ pela tabela

δ_1	δ_2	δ_3	regra	$\delta_1 + \delta_2 - \delta_3$	≤ 1?
1	1	1	V	1	sim
1	1	0	**F**	**2**	**não**
1	0	$*$	V	≤ 1	sim
0	1	$*$	V	≤ 1	sim
0	0	$*$	V	≤ 0	sim

A restrição é violada exatamente na única linha falsa ✓.

Cuidado: o verificador aceita, mas o solver não

O verificador avalia expressões Python sobre números 0/1, então aceita coisas como `a + b > 0` ou `a - b != 0`, que dão o resultado certo na tabela. Mas **nenhum solver de PL/PI aceita `>` estrito nem `!=`**. Treine escrever a versão que funcionaria no PuLP: `a + b >= 1` e `a + b == 1`.

9.2 3B – Carteira de investimentos

O problema. Cinco investimentos ($x_j = 1$ se investir em j), orçamento 14.

investimento j	1	2	3	4	5
retorno	5	6	4	10	10
custo	4	6	5	3	5

Regras: **R0** orçamento; **R1** no máximo 3 investimentos; **R2** investimento 1 **ou** 2; **R3 se** 2 **então** 1; **R4 se** 2, 3 **ou** 4 **então** 1; **R5 se** 1 **e** 5 **então** 3.

Traduzindo regra por regra

Regra	Lógica	Restrição
R0	orçamento	$4x_1 + 6x_2 + 5x_3 + 3x_4 + 5x_5 \leq 14$
R1	no máximo 3 de 5	$x_1 + x_2 + x_3 + x_4 + x_5 \leq 3$
R2	$P_1 \vee P_2$ (inclusivo)	$x_1 + x_2 \geq 1$
R3	$P_2 \Rightarrow P_1$	$x_2 \leq x_1$
R4	$P_2 \Rightarrow P_1$, $P_3 \Rightarrow P_1$, $P_4 \Rightarrow P_1$	(a) $x_2 + x_3 + x_4 \leq 3x_1$ ou (b) $x_j \leq x_1$, $j = 2,3,4$
R5	$P_1 \wedge P_5 \Rightarrow P_3$ ($k = 2$)	$x_1 + x_5 - x_3 \leq 1$

Por que (a) funciona para R4? Se $x_1 = 0$, então $x_2 + x_3 + x_4 \leq 0$ e nenhum dos três pode ser escolhido. Se $x_1 = 1$, então $x_2 + x_3 + x_4 \leq 3$, que sempre vale. O coeficiente 3 é o "M" (o maior valor possível da soma).

```python
def modelo_investimentos(desagregada=True):
    prob = pulp.LpProblem("investimentos", pulp.LpMaximize)
    x = {j: pulp.LpVariable(f"x{j}", cat="Binary") for j in J}
    prob += pulp.lpSum(retorno[j]*x[j] for j in J), "retorno"
    prob += pulp.lpSum(custo[j]*x[j] for j in J) <= ORC, "R0_orcamento"
    prob += pulp.lpSum(x[j] for j in J) <= 3, "R1_max3"
    prob += x[1] + x[2] >= 1, "R2_1_ou_2"
    prob += x[2] <= x[1], "R3_2_implica_1"
    if desagregada:                                    # R4 (b): uma por
    investimento
        for j in (2, 3, 4):
            prob += x[j] <= x[1], f"R4_{j}_implica_1"
    else:                                              # R4 (a): agregada
        prob += x[2] + x[3] + x[4] <= 3*x[1], "R4_agregada"
    prob += x[1] + x[5] - x[3] <= 1, "R5_1e5_implica_3"
    return prob, x
```

Note que o dicionário `x` é indexado de 1 a 5 (`J = range(1, 6)`), igual ao enunciado.

Resultado. Ótimo **21**, com os investimentos {1,2,4} (custo $4+6+3=13$). Sem R5, o ótimo seria 25 com {1,4,5}; R5 exige o 3 junto com 1 e 5, e com o 3 seriam 4 investimentos, o que viola R1.

Cuidado: diagnóstico pelo resultado

- Deu **26** com {2,4,5}? Faltam R3/R4: o investimento 2 está sem o 1.
- Deu **25** com {1,4,5}? Falta (ou está errada) a R5.
- Escreveu R5 como $x_1 + x_5 \leq x_3$? Errado: com $x_1 = 1$, $x_5 = 0$, isso exige $x_3 = 1$. A versão certa é $x_1 + x_5 - x_3 \leq 1$.

9.3 Experimento: agregada × desagregada

As duas versões de R4 têm **exatamente as mesmas soluções inteiras**. Qual é "melhor"?

	valor da relaxação	solução da relaxação
ótimo inteiro	21	{1,2,4}
agregada (a)	23,71	$x = (0{,}57;\ 0{,}43;\ 0{,}29;\ 1;\ 0{,}71)$
desagregada (b)	22,5	$x = (0{,}5;\ 0{,}5;\ 0{,}5;\ 0{,}5;\ 1)$

Explicação. Somando as três restrições de (b), $x_2 \leq x_1$, $x_3 \leq x_1$ e $x_4 \leq x_1$, obtemos $x_2 + x_3 + x_4 \leq 3x_1$, que é (a). Logo *todo* ponto que satisfaz (b) satisfaz (a): o poliedro de (b) está **dentro** do de (a), e (b) é uma formulação **mais forte** (Seção 3.4). A recíproca é falsa: o ponto $x_1 = \frac{1}{3}$, $x_2 = 1$, $x_3 = x_4 = 0$ satisfaz (a) ($1 \leq 1$) mas viola (b) ($1 \not\leq \frac{1}{3}$). Com x_1 pequeno, (a) ainda deixa um dos outros inteiro; (b) amarra cada um individualmente.

Lições do Bloco 3

- Menor limite superior = relaxação mais forte = menos nós no *branch-and-bound*.
- A formulação desagregada tem *mais* restrições e mesmo assim é *melhor*: "menos restrições" não significa "modelo melhor".
- Com (b), a R3 ($x_2 \leq x_1$) fica **redundante**, porque já aparece em R4. Com (a), ela ainda fortalece a relaxação.

Capítulo 10

Bloco 4 – Representação de valores discretos

Receita: variável que só assume valores de uma lista

Se x só pode valer $v_1, \ldots, v_k$ (por exemplo, {4,6,8,12,20,24}), use uma binária por valor:

$$x = \sum_{i=1}^{k} v_i \, y_i, \qquad \sum_{i=1}^{k} y_i = 1, \qquad y_i \in \{0,1\}.$$

A segunda restrição escolhe **exatamente um** valor, e a primeira "lê" qual foi escolhido.
Bônus: qualquer função de x também fica linear nos y_i:

$$\phi(x) = \sum_{i=1}^{k} \phi(v_i) \, y_i,$$

mesmo que ϕ seja não linear, porque $\phi(v_i)$ é só uma constante para cada i.

10.1 O problema: diâmetros de tubulação

Três trechos de tubulação, com comprimentos L_j, precisam transportar uma vazão mínima q_j. Cada trecho usa **um** diâmetro comercial. A vazão máxima de um tubo de diâmetro d é $0{,}5\,d^2$ L/s (**não linear** em d). O estoque de cada diâmetro é limitado. Queremos minimizar o custo total.

diâmetro d (cm)	4	6	8	12	20	24
preço (R$/m)	8	12	18	30	70	100
vazão máx. $0{,}5d^2$ (L/s)	8	18	32	72	200	288
estoque (m)	500	500	500	150	250	300

trecho j	1	2	3
L_j (m)	300	200	250
q_j (L/s)	15	60	150

Perguntas da discussão

1. *Por que não uma variável inteira d_j com $4 \le d_j \le 24$?* Porque 5, 7, 9, … não existem comercialmente, e preço e vazão não são funções lineares de d.
2. *A vazão cresce com d^2; como o modelo continua linear?* Os coeficientes $0{,}5v_i^2$ são **constantes** (8, 18, 32, …). O que varia são as binárias y_{jd}.

3. *Qual restrição acopla os trechos?* O **estoque**. Sem ele, cada trecho escolheria sozinho o diâmetro mais barato que dá vazão: (6,12,20), custo 27 100.

10.2 Construindo o modelo

Variáveis. $y_{jd} \in \{0,1\} = 1$ se o trecho j usa o diâmetro d (3 trechos $\times$ 6 diâmetros $= 18$ binárias).

Objetivo. O custo de usar d no trecho j é $L_j \cdot \text{preço}_d$ (constante!):

$$\min \sum_j \sum_d L_j \, \text{preço}_d \, y_{jd}.$$

Restrições.

$$\begin{array}{lll} \sum_d y_{jd} = 1 & \forall j & \text{(um diâmetro por trecho)} \\ \sum_d 0{,}5d^2 \, y_{jd} \geq q_j & \forall j & \text{(vazão: linear nos } y\text{)} \\ \sum_j L_j \, y_{jd} \leq \text{estoque}_d & \forall d & \text{(metros usados de cada diâmetro)} \end{array}$$

```
def modelo_tubos():
    prob = pulp.LpProblem("tubos", pulp.LpMinimize)
    y = {(j, d): pulp.LpVariable(f"y_{j}_{d}", cat="Binary") for j in range(3) for d
     in D}

    # TODO 1: custo total
    prob += pulp.lpSum(L[j]*preco[d]*y[j, d] for j in range(3) for d in D), "custo"
    for j in range(3):

        # TODO 2: exatamente um diâmetro por trecho
        prob += pulp.lpSum(y[j, d] for d in D) == 1, f"um_diametro_{j}"

        # TODO 3: vazão mínima (vazao[d] = 0.5*d**2 é um NÚMERO)
        prob += pulp.lpSum(vazao[d]*y[j, d] for d in D) >= q[j], f"vazao_{j}"

    # TODO 4: estoque de cada diâmetro
    for d in D:
        prob += pulp.lpSum(L[j]*y[j, d] for j in range(3)) <= estoque[d], f"estoque_
    {d}"
    return prob, y
```

10.3 Resultado e interpretação

Para entender a resposta, liste as opções que dão vazão suficiente em cada trecho:

trecho	diâmetros com vazão suficiente	custo de cada um
1 ($q = 15$, 300 m)	6, 8, 12, 20, 24	3 600; 5 400; 9 000; 21 000; 30 000
2 ($q = 60$, 200 m)	12, 20, 24	6 000; 14 000; 20 000
3 ($q = 150$, 250 m)	20, 24	17 500; 25 000

- O trecho 2 queria o diâmetro 12, mas só há 150 m em estoque e ele precisa de 200 m. Então o trecho 2 usa 20 ou 24.

- O diâmetro 20 tem 250 m: dá para o trecho 3 (250 m) *ou* para o trecho 2 (200 m), não para os dois.
- Opções: trecho 2 com 20 e trecho 3 com 24 custa $14\,000 + 25\,000 = 39\,000$; trecho 2 com 24 e trecho 3 com 20 custa $20\,000 + 17\,500 = 37\,500$. A segunda é melhor.

Ótimo: diâmetros $(6, 24, 20)$, custo $3\,600 + 20\,000 + 17\,500 = \mathbf{41\,100}$.

Cuidado: erros típicos

- Esquecer o estoque: dá 27 100 (a dica da verificação avisa).
- Usar ≤ 1 em "um diâmetro por trecho": o modelo pode não escolher nenhum, mas aí a vazão falha; use $= 1$, que é o que o problema diz.
- Escrever a vazão como `0.5*d**2` com `d` sendo uma variável do PuLP: não é linear. A potência é calculada *sobre os dados*, antes, no dicionário `vazao`.

Para pensar: variável semicontínua

Uma variável $x \in \{0\} \cup [\ell, u]$ ("ou desligada, ou entre ℓ e u") combina este bloco com o Bloco 1:

$$\ell\, y \leq x \leq u\, y, \qquad y \in \{0,1\}.$$

Com $y = 0$, $x = 0$; com $y = 1$, $\ell \leq x \leq u$.

Capítulo 11

Bloco 5 – "Ou exclusivo" e indicadores (TVL)

No Bloco 3, as proposições eram *variáveis* (δ_1, δ_2), e $P_1 \triangledown P_2$ virava simplesmente $\delta_1 + \delta_2 = 1$. Agora as proposições são **restrições**: queremos "exatamente uma entre $f(x) \leq 0$ e $g(x) \leq 0$ vale". Para isso precisamos de binárias que *indiquem* se cada restrição vale, nos dois sentidos.

11.1 A receita da teoria de variáveis lógicas (TVL)

Receita: indicador $\delta \leftrightarrow f(x) \leq 0$, com $m \leq f(x) \leq M$ no domínio

O "se e somente se" tem uma **ida** e uma **volta**:

- **Ida** – $\delta = 1 \Rightarrow f(x) \leq 0$: $\quad f(x) \leq M(1-\delta)$.
- **Volta** – $\delta = 0 \Rightarrow f(x) > 0$. O solver não trabalha com ">", então usamos $f(x) \geq \varepsilon$, onde $\varepsilon > 0$ é a menor folga que importa:

$$\varepsilon - f(x) - (\varepsilon - m)\,\delta \leq 0.$$

Para o "ou exclusivo" entre duas restrições: crie $\delta_1 \leftrightarrow f \leq 0$ e $\delta_2 \leftrightarrow g \leq 0$ (ida + volta de cada uma) e acrescente $\delta_1 + \delta_2 = 1$.

De onde vem a volta? Queremos "$\delta = 0 \Rightarrow \varepsilon - f(x) \leq 0$". É o Teorema 4.1 aplicado à função $h(x) = \varepsilon - f(x)$: a restrição é $h(x) - M_h\delta \leq 0$, com $M_h \geq \max h = \varepsilon - \min f = \varepsilon - m$. Substituindo, chegamos a $\varepsilon - f(x) - (\varepsilon - m)\delta \leq 0$. Conferindo pelos casos:

- $\delta = 0$: fica $\varepsilon - f(x) \leq 0$, isto é, $f(x) \geq \varepsilon$ ✓;
- $\delta = 1$: fica $m - f(x) \leq 0$, isto é, $f(x) \geq m$, que sempre vale ✓(desligada).

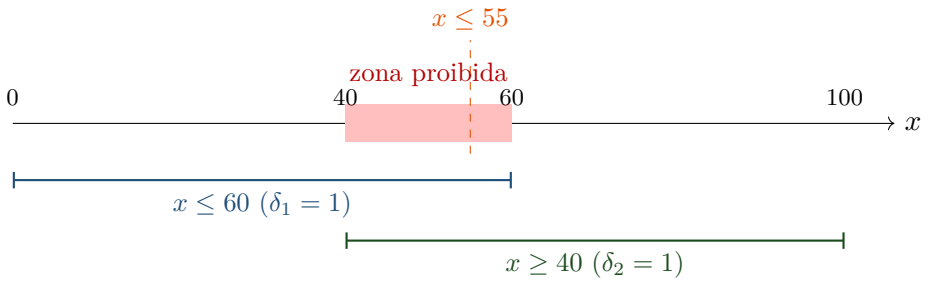
Cuidado: notação dos slides

Nos slides, a volta aparece como "$\epsilon - f(x) - m_1\delta_1 \leq 0$", onde m_1 faz o papel de $(\varepsilon - \min f)$. O notebook e esta apostila escrevem $(\varepsilon - m)$ explicitamente, com $m = \min f$, para evitar ambiguidade. Cuidado ao misturar as duas notações.

11.2 5A – Zona de ressonância

O problema. A velocidade $x \in \{0,1,\ldots,100\}$ de um motor **não pode ficar na zona de ressonância** $40 \le x \le 60$. Há ainda um limite térmico $x \le 55$. Maximizar x.

A lógica. "Fora da zona" é $(x \le 60) \triangledown (x \ge 40)$: exatamente uma das duas vale.



Para $x < 40$, só a primeira vale; para $x > 60$, só a segunda; entre 40 e 60, as duas valem, e o "exclusivo" proíbe.

Perguntas da discussão

- *Qual o ótimo?* Com $x \le 55$, não dá para pular para $x \ge 61$. Fora da zona e abaixo de 55, o melhor é $x = 39$.
- *As regiões $x \le 60$ e $x \ge 40$ se sobrepõem. Usando só a "ida", o modelo proibiria $x = 55$?* Não, como o experimento vai mostrar.

Exercício 5A-1: a função `indicador`

A função recebe o valor (ou a expressão) de f, o indicador d, os limites M e m e o ε, e devolve a **lista** com as duas restrições:

```
def indicador(f, d, M, m, eps):

    """Restrições que impõem  d = 1  <=>  f <= 0."""
    ida   = f <= M*(1 - d)                  # d = 1  =>  f <= 0
    volta = eps - f - (eps - m)*d <= 0      # d = 0  =>  f >= eps
    return [ida, volta]
```

O detalhe elegante é que a **mesma função** serve para números e para o PuLP. O verificador chama `indicador` com números ($f(x)$ calculado, $d \in \{0,1\}$), e a comparação `<=` devolve `True`/`False`. No modelo, `f` e `d` são expressões/variáveis do PuLP, e o `<=` devolve uma restrição. O verificador testa todos os $x \in \{0,\ldots,100\}$ e $d \in \{0,1\}$ (202 pares) e exige: as restrições valem $\iff$ $(d = 1 \iff f(x) \le 0)$.

Exercício 5A-2: o modelo

Limites. Para $x \in [0,100]$: $f_1 = x - 60 \in [-60, 40]$ e $f_2 = 40 - x \in [-60, 40]$. Logo $M_1 = M_2 = 40$ e $m_1 = m_2 = -60$. Como x é inteiro, "$f > 0$" equivale a "$f \ge 1$": $\varepsilon = 1$.

```
EPS = 1
def modelo_ressonancia(com_volta=True):
    prob = pulp.LpProblem("ressonancia", pulp.LpMaximize)
    x  = pulp.LpVariable("x", lowBound=0, upBound=100, cat="Integer")
    d1 = pulp.LpVariable("d1", cat="Binary")      # d1 = 1 <=> x <= 60
    d2 = pulp.LpVariable("d2", cat="Binary")      # d2 = 1 <=> x >= 40
    prob += 1*x, "velocidade"
    prob += x <= 55, "limite_termico"
    f1 = x - 60
    f2 = 40 - x
    M1, m1 = 40, -60                              # TODO 1
    M2, m2 = 40, -60
    for delta, f_, M_, m_ in ((d1, f1, M1, m1), (d2, f2, M2, m2)):   # TODO 2
        restr = indicador(f_, delta, M_, m_, EPS)

        for c in (restr if com_volta else restr[:1]):   # sem volta: só a ida
            prob += c
    prob += d1 + d2 == 1, "xor"                   # TODO 3
    return prob, x, d1, d2
```

O que essas restrições dizem, em português. Substituindo os números:

	ida	volta
δ_1	$x - 60 \le 40(1-\delta_1)$: se $\delta_1 = 1$, $x \le 60$	$x \ge 61 - 61\delta_1$: se $\delta_1 = 0$, $x \ge 61$
δ_2	$x \ge 40\,\delta_2$: se $\delta_2 = 1$, $x \ge 40$	$x \le 39 + 61\delta_2$: se $\delta_2 = 0$, $x \le 39$

Com $\delta_1 + \delta_2 = 1$ há dois casos. Se $(\delta_1,\delta_2) = (1,0)$, então $x \le 60$ e $x \le 39$, ou seja, $x \le 39$. Se $(\delta_1,\delta_2) = (0,1)$, então $x \ge 61$ e $x \ge 40$, ou seja, $x \ge 61$, o que é incompatível com $x \le 55$. O ótimo é **39**.

11.3 Experimento: sem a volta

Só com as idas, o solver devolve **55**, *dentro* da zona proibida. Ele escolhe $\delta_1 = 1$ ($x \le 60$ ✓) e $\delta_2 = 0$. Sem a volta, $\delta_2 = 0$ **não tem consequência nenhuma**: o modelo não "sabe" que $\delta_2 = 0$ deveria obrigar $x < 40$. Com $x = 55$, as duas proposições são verdadeiras, violando o "exclusivo", mas o modelo não percebe. A volta ($\delta_2 = 0 \Rightarrow x \le 39$) fecha essa brecha.

11.4 5B (desafio) – Desconto por faixa, onde a volta *não* é necessária

O problema. Um fornecedor cobra R$ 10/kg se $x \le 99$ kg e R$ 8/kg (sobre *todos* os kg) se $x \ge 100$ kg. Preciso de pelo menos D kg (x inteiro, até $U = 300$). Minimizar o custo.

Perguntas da discussão

1. *Para $D = 90$: comprar 90 kg ou 100 kg?* $90 \times 10 = 900$ contra $100 \times 8 = 800$: compensa comprar **100 kg**. O ponto de indiferença é $10D = 800$, ou seja, $D = 80$. A partir de 80 kg, vale comprar 100.
2. *O custo é $10\,x\,\delta_1 + 8\,x\,\delta_2$, produto de variável por binária. Como linearizar?* Com variáveis auxiliares $u_1 = x\delta_1$ e $u_2 = x\delta_2$, mais $u_1 + u_2 = x$, $u_1 \le U\delta_1$ e $u_2 \le U\delta_2$. Como exatamente

um δ vale 1, o u correspondente recebe todo o x e o outro é zero.

3. *Precisamos da volta?* Não; veja abaixo.

Indicadores. $\delta_1 \leftrightarrow x \le 99$, com $f_1 = x - 99 \in [-99, 201]$; $\delta_2 \leftrightarrow x \ge 100$, com $f_2 = 100 - x \in [-200, 100]$.

```
def modelo_desconto(D, U=300, Q=100, EPS=1):
    prob = pulp.LpProblem("desconto", pulp.LpMinimize)
    x  = pulp.LpVariable("x", lowBound=0, upBound=U, cat="Integer")

    d1 = pulp.LpVariable("d1", cat="Binary")      # d1 = 1 <=> x <= Q-1 (preço cheio
    )

    d2 = pulp.LpVariable("d2", cat="Binary")      # d2 = 1 <=> x >= Q   (desconto)
    u1 = pulp.LpVariable("u1", lowBound=0)        # u1 = x*d1
    u2 = pulp.LpVariable("u2", lowBound=0)        # u2 = x*d2
    prob += 10*u1 + 8*u2, "custo"
    prob += x >= D, "demanda"
    prob += d1 + d2 == 1, "xor"                                    # TODO 1
    f1 = x - (Q - EPS); M1, m1 = U - (Q - EPS), -(Q - EPS)         # TODO 2
    f2 = Q - x;         M2, m2 = Q, Q - U
    for c in indicador(f1, d1, M1, m1, EPS) + indicador(f2, d2, M2, m2, EPS):
        prob += c
    prob += u1 + u2 == x, "soma_u"                                 # TODO 3
    prob += u1 <= U*d1, "u1_ativa"
    prob += u2 <= U*d2, "u2_ativa"
    return prob, x
```

D (kg)	50	75	78	80	81	90	130
custo ótimo (R$)	500	750	780	800	800	800	1040

Ou seja, custo$(D) = \min\{10D$ (se $D \le 99)$, $8\max(D,100)\}$.

Por que a volta é dispensável aqui? As regiões $x \le 99$ e $x \ge 100$ **particionam** o domínio: todo x está em exatamente uma. Com $\delta_1 + \delta_2 = 1$, "$\delta_2 = 0$" é o mesmo que "$\delta_1 = 1$", e a *ida* de δ_1 ($x \le 99$) já faz o papel da volta de δ_2. Não há como "trapacear" escolhendo o preço com desconto para $x < 100$. Na ressonância, as regiões se sobrepunham (entre 40 e 60 as duas valem), e por isso a volta era indispensável. *Teste:* retire a segunda restrição de `indicador` em `modelo_desconto`; os custos não mudam.

11.5 5C – O produto de binárias $\delta_3 = \delta_1\delta_2$

O produto de duas binárias é 1 só quando as duas são 1, ou seja, $\delta_3 = 1 \leftrightarrow (\delta_1 = 1 \wedge \delta_2 = 1)$, que é o mesmo que $\delta_3 = 1 \leftrightarrow \delta_1 + \delta_2 = 2$. Cada sentido vira uma restrição pelo Teorema:

- ($\Rightarrow$) $\delta_3 = 1 \Rightarrow 2 - \delta_1 - \delta_2 \le 0$. O indicador da implicação é $(1 - \delta_3)$ e $\max(2 - \delta_1 - \delta_2) = 2$:

$$2 - \delta_1 - \delta_2 - 2(1 - \delta_3) \le 0 \implies -\delta_1 - \delta_2 + 2\delta_3 \le 0.$$

- ($\Leftarrow$) pela contrapositiva, $\delta_3 = 0 \Rightarrow \delta_1 + \delta_2 - 1 \le 0$, com $\max(\delta_1 + \delta_2 - 1) = 1$:

$$\delta_1 + \delta_2 - 1 - \delta_3 \le 0 \implies \delta_1 + \delta_2 - \delta_3 \le 1.$$

δ_1	δ_2	δ_3	$\delta_3 = \delta_1\delta_2$?	$-\delta_1 - \delta_2 + 2\delta_3 \le 0$	$\delta_1 + \delta_2 - \delta_3 \le 1$
0	0	0	V	✓	✓
0	0	1	F	×	✓
0	1	0	V	✓	✓
0	1	1	F	×	✓
1	0	0	V	✓	✓
1	0	1	F	×	✓
1	1	0	F	✓	×
1	1	1	V	✓	✓

As duas restrições juntas são satisfeitas exatamente nas linhas verdadeiras ✓. No verificador (no notebook, os nomes usam letras gregas):

```
("d3 = d1*d2   (e)",  3, lambda a, b, c: c == (a and b),
     lambda a, b, c: (-a - b + 2*c <= 0) and (a + b - c <= 1)),
("d3 = d1 ou d2 (ou)", 3, lambda a, b, c: c == (a or b),
     lambda a, b, c: (c >= a) and (c >= b) and (c <= a + b)),
```

O "ou" (desafio). $\delta_3 = \delta_1 \vee \delta_2$: se algum é 1, então $\delta_3 = 1$ ($\delta_3 \ge \delta_1$, $\delta_3 \ge \delta_2$); se os dois são 0, então $\delta_3 = 0$ ($\delta_3 \le \delta_1 + \delta_2$).

Cuidado: conexão com o Bloco 3

$-\delta_1 - \delta_2 + 2\delta_3 \le 0$ é a soma de $\delta_3 \le \delta_1$ e $\delta_3 \le \delta_2$: é a versão *agregada*! As duas restrições separadas são equivalentes nos binários e formam uma relaxação mais forte. Ambas passam no verificador.
Observação sobre os slides: no slide "Outro exemplo", a primeira linha traz $\delta_1 = 1 \vee \delta_2 = 1$, mas a derivação que segue é a da **conjunção** ($\wedge$), que é a correta para o produto.

Receita: produto de contínua por binária: $u = x\,\delta$, com $0 \le x \le U$

$$u \le U\delta, \qquad u \le x, \qquad u \ge x - U(1-\delta), \qquad u \ge 0.$$

Se $\delta = 0$: $u \le 0$, logo $u = 0$. Se $\delta = 1$: $u \le x$ e $u \ge x$, logo $u = x$. (No 5B bastaram duas restrições porque $u_1 + u_2 = x$ e $\delta_1 + \delta_2 = 1$ já fazem o resto.)

Cuidado: erros típicos do Bloco 5

- Na volta, usar m no lugar de $(\varepsilon - m)$; ou trocar o sinal de f. O verificador mostra um contraexemplo (x,δ); refaça os dois casos ($\delta = 0$ e $\delta = 1$) à mão.
- Calcular M e m errados: lembre que $f_2 = 40 - x$ é *decrescente* em x. O máximo ocorre em $x = 0$ e o mínimo em $x = 100$.
- Com variáveis contínuas, ε é uma escolha de modelagem (tolerância). Valores muito pequenos causam problemas numéricos.

Capítulo 12

Bloco 6 – Problema da mochila binária

Temos itens j com **valor** c_j e **peso** a_j, e uma mochila de capacidade b. Queremos o conjunto de maior valor total que cabe. A mochila é um dos problemas clássicos mais importantes: aparece sozinha (orçamento de projetos, carga de contêineres) e como *subproblema* de modelos maiores.

12.1 Quatro versões do problema

Caso	Fracionável?	Limite por item	Tipo	Solução
1	sim	ilimitado	PL	trivial: só o item de maior c_j/a_j, $x_{j^*} = b/a_{j^*}$
2	não	ilimitado	PI	**não trivial**
3	sim	≤ 1	PL	gulosa por c_j/a_j, fracionando o último item
4	**não**	$\leq \mathbf{1}$	**PB**	**não trivial** ← *este bloco*

Exemplo prático: caso 3 nos slides: a gulosa fracionária

$\max 6x_1+8x_2+4x_3+x_4$ s.a. $x_1+2x_2+2x_3+x_4 \leq 4$, $0 \leq x_j \leq 1$. Razões: 6, 4, 2, 1. Pegamos $x_1 = 1$ (sobram 3), $x_2 = 1$ (sobra 1) e $x_3 = 1/2$ (acabou); $x_4 = 0$. Valor $6 + 8 + 2 = 16$.

O caso 3 é exatamente a **relaxação linear** do caso 4. Então a gulosa fracionária dá, sem simplex nenhum, o limite superior da mochila binária.

Outros autores: Dantzig (1957) e o "item crítico"

Esse resultado é clássico: Dantzig mostrou em 1957 que a relaxação contínua da mochila é resolvida ordenando os itens por c_j/a_j e enchendo a mochila até o primeiro item que não cabe inteiro, o **item crítico**, que entra fracionado [22, 23]. O valor resultante (o "limite de Dantzig") é o limite superior usado nos primeiros algoritmos de *branch-and-bound* para a mochila.

12.2 O problema do notebook: carteira de projetos

Dez projetos, orçamento $B = 74$ (R$ mil):

projeto j	1	2	3	4	5	6	7	8	9	10
valor c_j	29	47	14	33	37	13	41	36	38	16
custo a_j	34	25	19	32	18	12	16	36	16	16
c_j/a_j	0,85	1,88	0,74	1,03	2,06	1,08	**2,56**	1,00	2,38	1,00

Perguntas da discussão

1. *Variável? Por que binária e não inteira qualquer?* $x_j = 1$ se o projeto j é financiado. Um projeto não pode ser financiado "duas vezes" nem pela metade: é o caso 4.
2. *Objetivo e restrição?* $\max \sum_j c_j x_j$ s.a. $\sum_j a_j x_j \leq 74$. Uma única restrição!
3. *Qual entra primeiro pela razão? A gulosa resolve? E a relaxação, é o caso 3?* O projeto 7 (razão 2,56) entra primeiro. A gulosa *não* garante o ótimo. A relaxação é o caso 3.

Exercício 6A

```
def modelo_mochila(valor, peso, B, extras=False):
    prob = pulp.LpProblem("mochila", pulp.LpMaximize)
    x = {j: pulp.LpVariable(f"x_{j}", cat="Binary") for j in valor}
    prob += pulp.lpSum(valor[j]*x[j] for j in valor), "valor_total"        # TODO 1
    prob += pulp.lpSum(peso[j]*x[j] for j in valor) <= B, "capacidade"     # TODO 2
    if extras:

        ...                                                                # 6B,

    abaixo
    return prob, x
```

`valor` e `peso` são dicionários com chaves $1, \ldots, 10$ (`enumerate(..., start=1)`), então `for j in valor` percorre os projetos.

12.3 Comparando quatro respostas

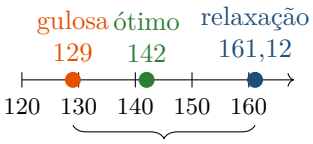
Gulosa (heurística). Ordem por razão: 7, 9, 5, 2, 6, 4, 8, 10, 1, 3.

passo	projeto	custo acumulado	valor	decisão
1	7	16	41	entra
2	9	32	79	entra
3	5	50	116	entra
4	2	75	–	não cabe ($75 > 74$)
5	6	62	129	entra
6+	4, 8, 10, 1, 3	–	–	nenhum cabe nos 12 restantes

Resultado: $\{5,6,7,9\}$, valor **129**.

Relaxação linear (Dantzig). Mesma ordem, mas o projeto 2 entra *fracionado* para usar os 24 que sobram: $x_2 = 24/25 = 0{,}96$. Valor $116 + 0{,}96 \cdot 47 =$ **161,12**. O projeto 2 é o item crítico.

Ótimo (PuLP/CBC) e programação dinâmica. $\{2,7,9,10\}$, custo $25+16+16+16=73$, valor $47+41+38+16=\mathbf{142}$. A programação dinâmica $O(nB)$ do notebook confirma (você verá PD mais adiante na disciplina).



intervalo que o *branch-and-bound* precisa fechar

12.4 6B – Mochila com regras de negócio

Juntando os blocos 1 a 3:

Regra	Lógica	Restrição
(a) projetos 2 e 7 incompatíveis	exclusão mútua	$x_2 + x_7 \leq 1$
(b) se 9 então 5	implicação	$x_9 \leq x_5$
(c) no máximo 4 projetos	cardinalidade	$\sum_j x_j \leq 4$
(d) 3 ou 6 (ou ambos)	ou inclusivo	$x_3 + x_6 \geq 1$

```
if extras:
    prob += x[2] + x[7] <= 1, "excl_2_7"          # (a)
    prob += x[9] <= x[5], "c9_implica_5"           # (b)
    prob += pulp.lpSum(x.values()) <= 4, "max4"    # (c)
    prob += x[3] + x[6] >= 1, "c3_ou_6"            # (d)
```

Resultado. Ótimo **135** com $\{2,5,6,9\}$ (custo $25+18+12+16=71$). A regra (a) corta a solução de 6A (que tinha 2 e 7 juntos); (b) obriga o 5 a acompanhar o 9; (d) é satisfeita pelo 6.

Cuidado: erros típicos

- Escrever (b) como $x_5 \leq x_9$: isso é "se 5 então 9", a implicação ao contrário.
- Escrever (d) como $x_3 + x_6 = 1$: isso é o "ou exclusivo" e proíbe escolher os dois.

Reflexão do notebook

A gulosa fica em 129, a relaxação diz "até 161" e o ótimo é 142. Quem fecha a diferença é o *branch-and-bound*, que usa a relaxação como limite e ramifica nas variáveis fracionárias (aqui, primeiro em x_2).

Capítulo 13

Bloco 7 – *Bin packing*

Empacotar **todos** os itens no **menor número de caixas** de capacidade b. É a mochila "ao contrário": agora todo item entra, e o que queremos minimizar é o número de caixas.

Os dados. Dez itens de tamanhos $w = (60,36,53,51,58,48,47,48,53,35)$, caixas de capacidade $b = 100$. A soma dos tamanhos é 489, então o **limite de volume** é $\lceil 489/100 \rceil = 5$ caixas.

Perguntas da discussão

1. *Quais variáveis?* $x_{ij} = 1$ se o item j vai para a caixa i; $y_i = 1$ se a caixa i é usada.
2. *Quantas caixas considerar?* No pior caso, uma por item: n caixas ($i = 1,\ldots,n$).
3. *As duas famílias de restrições?* Todo item vai para **exatamente uma** caixa, e a capacidade é respeitada, **mas só se a caixa for usada**. "Só se usada" é o Bloco 1 de novo: $\sum_j w_j x_{ij} \le b\, y_i$.
4. *Dá para usar 5 caixas?* Não; veja o argumento abaixo.

13.1 O modelo

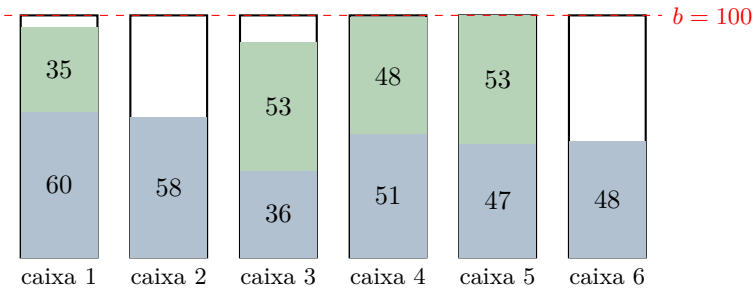
$$
\begin{aligned}
\min \quad & \sum_{i=1}^{n} y_i \\
\text{s.a.} \quad & \sum_{i=1}^{n} x_{ij} = 1 && \forall j \quad \text{(todo item em exatamente uma caixa)} \\
& \sum_{j=1}^{n} w_j x_{ij} \le b\, y_i && \forall i \quad \text{(capacidade, só se a caixa é usada)} \\
& x_{ij}, y_i \in \{0,1\}.
\end{aligned}
$$

Repare que $b\, y_i$ faz dois papéis ao mesmo tempo: se $y_i = 1$, limita a ocupação a b; se $y_i = 0$, proíbe colocar qualquer item na caixa i (é uma restrição de ativação com $M = b$).

```python
def modelo_binpacking(w, b, simetria=False):
    n = len(w)
    prob = pulp.LpProblem("binpacking", pulp.LpMinimize)
    y = {i: pulp.LpVariable(f"y_{i}", cat="Binary") for i in range(n)}
    x = {(i, j): pulp.LpVariable(f"x_{i}_{j}", cat="Binary") for i in range(n) for j
     in range(n)}
    prob += pulp.lpSum(y.values()), "n_caixas"                                    #
    TODO 1
    for j in range(n):                                                            #
    TODO 2
        prob += pulp.lpSum(x[i, j] for i in range(n)) == 1, f"item_{j}_em_uma_caixa"
    for i in range(n):                                                            #
    TODO 3
        prob += pulp.lpSum(w[j]*x[i, j] for j in range(n)) <= b*y[i], f"capacidade_{
    i}"
    if simetria:                                                                  #

    desafio
        for i in range(n-1):
            prob += y[i] >= y[i+1], f"simetria_{i}"
    return prob, x, y
```

13.2 Resultado: 6 caixas



Ocupações: 95, 58, 89, 99, 100 e 48. A heurística FFD (*First-Fit Decreasing*) do notebook também usa 6 caixas, e a relaxação linear vale 4,89.

Por que é impossível usar 5 caixas? Um argumento curto, só com aritmética:

1. Há **cinco itens maiores que 50**: 60, 58, 53, 53 e 51. Dois deles nunca cabem juntos, então cada um ocupa uma caixa diferente. Com 5 caixas, cada caixa teria exatamente um item grande.
2. As folgas dessas caixas seriam 40, 42, 47, 47 e 49.
3. Os itens pequenos são 48, 48, 47, 36 e 35. Dois pequenos juntos somam pelo menos $35 + 36 = 71 > 49$, então cada caixa recebe *no máximo um* pequeno.
4. Os **dois itens de tamanho 48** precisam de folga ≥ 48, mas só a caixa do 51 tem folga 49. Um dos 48 fica sem lugar.

Logo são necessárias pelo menos 6 caixas, e o solver encontrou 6: **6 é ótimo**.

Outros autores: limitantes para o *bin packing*

O passo 1 do argumento é a ideia por trás dos limitantes inferiores clássicos de Martello e Toth: itens maiores que metade da capacidade não podem dividir caixa entre si e precisam ser contados separadamente, o que melhora o limite de volume $\lceil \sum w_j/b \rceil$ [23, 27]. É um exemplo de como raciocínio combinatório complementa a relaxação linear.

13.3 Reflexões: relaxação fraca e simetria

Relaxação fraca. Na relaxação, o solver pode usar $y_i = \sum_j w_j x_{ij}/b$, "meia caixa" aqui e ali, e o valor fica exatamente $\sum_j w_j/b = 4{,}89$. A relaxação não "enxerga" que itens grandes não se combinam.

Outros autores: a formulação "de Kantorovich" e alternativas

O modelo com x_{ij} e y_i é atribuído a Kantorovich [24]. Os surveys de Valério de Carvalho [26] e de Delorme, Iori e Martello [27] registram que sua relaxação linear é fraca (vale apenas o limite de volume) e que formulações baseadas em *padrões de corte*, como a de Gilmore e Gomory [25], têm relaxações muito mais fortes, ao custo de um número exponencial de variáveis.

Simetria. As caixas são **idênticas**: trocar os rótulos de duas caixas gera uma solução "diferente" para o solver, mas igual na prática. Uma solução com k caixas tem $k!$ cópias equivalentes, e o *branch-and-bound* pode perder tempo explorando todas.

Quebra de simetria. Exigir que as caixas sejam usadas *em ordem*:

$$y_i \geq y_{i+1} \qquad (i = 1, \ldots, n-1).$$

Isso não muda o ótimo (sempre dá para renumerar as caixas usadas como $1,2,\ldots,k$), mas elimina as cópias. No experimento do notebook com 14 itens, sem a quebra de simetria o CBC bateu o limite de 30 s; com ela, resolveu em 0,15 s (ambos com 6 caixas). Os tempos variam com máquina e solver, mas a tendência é clara.

Cuidado: erros típicos do Bloco 7

- Escrever a capacidade como $\sum_j w_j x_{ij} \leq b$ (sem y_i): nada liga y aos itens, e o solver zera todos os y_i, devolvendo "0 caixas".
- Usar $\sum_i x_{ij} \leq 1$ no lugar de $= 1$: o modelo "descarta" itens e usa 0 caixas.
- Com `timeLimit`, o PuLP pode reportar "Optimal" para uma solução que é apenas viável. Desconfie de tempos que coincidem com o limite.

Capítulo 14

Quadro-resumo: as receitas de linearização

Quero dizer…	Variáveis	Restrição(ões) linear(es)
$x > 0 \Rightarrow y = 1$ (custo fixo, ativação)	$y \in \{0,1\}$	$x \le My$
$y = 1 \Rightarrow f(x) \le 0$	y	$f(x) \le M(1-y)$
$y = 1 \Rightarrow x \ge m$ (lote mínimo)	y	$x \ge m\,y$
$x \in \{0\} \cup [\ell,u]$ (semicontínua)	y	$\ell y \le x \le uy$
$f(x) \le 0$ **ou** $g(x) \le 0$	y	$f \le M_1(1-y)$ e $g \le M_2\,y$
tarefas j,k não se sobrepõem	y_{jk}	$s_j + p_j \le s_k + M(1-y_{jk})$ e $s_k + p_k \le s_j + My_{jk}$
$P_1 \Rightarrow P_2$	δ_i	$\delta_1 \le \delta_2$
$P_1 \wedge P_2 \Rightarrow P_3$	δ_i	$\delta_1 + \delta_2 - \delta_3 \le 1$
$P_1 \wedge \cdots \wedge P_k \Rightarrow P_{k+1} \vee \cdots \vee P_n$	δ_i	$\sum_{i\le k}\delta_i - \sum_{i>k}\delta_i \le k-1$
no máximo / exatamente / pelo menos k de n	δ_i	$\sum \delta_i \le k$ / $= k$ / $\ge k$
$x \in \{v_1,\ldots,v_k\}$	y_i	$x = \sum v_i y_i,\ \sum y_i = 1$ (e $\phi(x) = \sum \phi(v_i) y_i$)
$\delta \leftrightarrow f(x) \le 0$	δ	$f \le M(1-\delta)$ e $\varepsilon - f - (\varepsilon - m)\delta \le 0$
$P_1 \triangledown P_2$ (com $\delta_i \leftrightarrow$ restrições)	δ_1,δ_2	indicadores de cada uma $+\ \delta_1 + \delta_2 = 1$
$\delta_3 = \delta_1\delta_2$	δ_3	$\delta_3 \le \delta_1,\ \delta_3 \le \delta_2$ (ou $-\delta_1 - \delta_2 + 2\delta_3 \le 0$), $\delta_1 + \delta_2 - \delta_3 \le 1$
$\delta_3 = \delta_1 \vee \delta_2$	δ_3	$\delta_3 \ge \delta_1,\ \delta_3 \ge \delta_2,\ \delta_3 \le \delta_1 + \delta_2$
$u = x\,\delta$ $(0 \le x \le U)$	$u \ge 0$	$u \le U\delta,\ u \le x,\ u \ge x - U(1-\delta)$

Boas práticas (que voltam em *branch-and-bound* e planos de corte)

1. M **o menor possível.** Calcule-o com os limites das variáveis ou com uma pequena PL. M grande enfraquece a relaxação e gera instabilidade numérica; M pequeno demais deixa o modelo errado.
2. **Formulações mais fortes** (desagregadas) podem valer mais que formulações "enxutas".
3. **Simetria atrapalha.** Quebre-a com restrições de ordenação ($y_i \ge y_{i+1}$).
4. **Modelo certo $\neq$ modelo bom.** Compare as **relaxações lineares** das alternativas.
5. **Teste os casos.** Para cada restrição com binária, substitua 0 e 1 e leia o que sobra. Se alguma leitura não fizer sentido, a restrição está errada.

Checklist antes de rodar o solver

☐ Cada variável tem o tipo certo (`Continuous`, `Integer`, `Binary`) e `lowBound=0` quando não pode ser negativa?

☐ O objetivo inclui *todos* os custos/lucros, inclusive os fixos ($s_i y_i$)?

☐ Toda binária está "amarrada" a algo? (Uma binária que não aparece em nenhuma restrição o solver escolhe livremente.)

☐ Usei $= 1$ onde o enunciado diz "exatamente um" e ≥ 1 onde diz "pelo menos um"?

☐ Nenhum `<`, `>`, `!=` nem produto de variáveis?

☐ Os nomes das restrições são únicos (índice na f-string)?

☐ O resultado faz sentido? Confira a solução contra o enunciado, restrição por restrição.

Ponte para a lista de prática

Os exercícios da lista (*Pratica_Exerc___Programacao_Inteira*) usam as técnicas deste notebook com problemas diferentes:

Exercício	Técnica(s)
1 – Artesão	Bloco 1: lote mínimo *se* produzir; custo fixo
2 – Máquinas paralelas	Blocos 6–7: alocação de tarefas a máquinas com objetivo *min-max*
3 – Tabuleiro 4×4	lógica + variável inteira auxiliar (paridade: $\sum = 2k$)
4 – *Flowshop* permutacional	Bloco 2: disjuntivas de ordenação (*big-M*)

Apêndice A

Respostas dos exercícios de fixação

Capítulo 1

1.1 (a) PB (só binárias); (b) PLI (quantidades inteiras e moldes binários, sem contínuas); (c) PLIM (contínuas e uma binária).

1.2 (a) contínua basta; (b) inteira; (c) binária; (d) contínua basta.

1.3 A PL continua valendo 1000 ($o = 5/3$). Inteiras: $o = 0$,$v = 5$ custa 1500; $o = 1$,$v = 2$ custa 1200; $o = 2$,$v = 0$ custa 1200. O ótimo é 1200 (duas soluções), e arredondar $o = 1{,}67 \to 2$ agora *acerta*. Moral: às vezes arredondar funciona, mas não há garantia.

Capítulo 2

2.1 $50 \cdot 3 + 31 \cdot 3 = 243 \leq 250$ e $3 \cdot 3 - 2 \cdot 3 = 3 \geq -4$: viável, valor $3 + 1{,}92 = 4{,}92$. Não é "vizinho" porque os vizinhos do ótimo da PL têm $x_1 \in \{1{,}2\}$ e $x_2 \in \{4{,}5\}$.

2.2 A gulosa pega o item 1 (peso 3) e depois o item 2 (peso 2): peso 5, valor 19. A enumeração confirma que (1,1,0), com valor 19, é ótimo. A gulosa acerta neste caso, por sorte.

2.3 Quarta-feira ($j = 3$) é coberta por quem começou em qua, ter, seg, dom e sáb: $x_0 + x_1 + x_2 + x_3 + x_6 \geq 19$.

Capítulo 3

3.1 Em minimização, a relaxação é limite inferior e a incumbente é limite superior: o ótimo está em [40,50]. Então 38 não pode, 45 pode e 52 não pode (já temos uma solução de valor 50). Gap $= (50 - 40)/50 = 20\%$.

3.2 Essa solução é **ótima** para o problema binário.

3.3 A relaxação desse nó já é inteira ($x_1 = 1$), com valor $12 \leq 13$: não há o que ramificar e ela é pior que a incumbente. O nó é podado.

Capítulo 4

4.1 $10y \leq x \leq 50y,\ \ y \in \{0{,}1\}$.

4.2 Queremos $\delta = 0 \Rightarrow f \geq 0$ com $f = x_1 - x_2 - 2$, cujo mínimo é $0 - 8 - 2 = -10$. Restrição: $x_1 - x_2 - 2 \geq -10\delta$. Teste: $\delta = 0$ dá $x_1 - x_2 \geq 2$ ✓; $\delta = 1$ dá $x_1 - x_2 \geq -8$, que sempre vale ✓.

4.3 $P_1 \Rightarrow \neg P_2 \equiv \neg P_1 \vee \neg P_2$, que vira $(1-\delta_1)+(1-\delta_2) \geq 1$, ou seja, $\delta_1 + \delta_2 \leq 1$. Só a linha (1,1) é proibida ✓.

4.4 $f_i(x) \leq M_i(1-\delta_i)$ para $i = 1,2,3$, e $\delta_1 + \delta_2 + \delta_3 \geq 2$.

4.5 (1) A relaxação fica fraca, e o *branch-and-bound* demora muito mais; (2) problemas numéricos: uma binária "quase zero" (dentro da tolerância de integralidade) multiplicada por 10^9 libera valores grandes de x, o fenômeno que a Gurobi chama de *trickle flow* (Seção 4.4).

Capítulo 5

5.1 Veja a função `modelo_wolsey` no Capítulo 6.

Apêndice B

Gabarito de valores ótimos do notebook

Valores do dicionário `ESPERADO` do notebook, para conferir sem rodar a verificação.

Chave	Valor	Onde
`wolsey_lp` / `wolsey_ip`	5,0984 / 5	Bloco 0
`fabrica` / `fabrica_regra`	1875 / 1375	Bloco 1
`M1` / `M2` / `disjuncao`	120 / 20 / 100	Bloco 2A
`1maq` / `1maq_release`	127 / 135	Bloco 2B
`investimentos`	21	Bloco 3B
`inv_rel_agregada` / `inv_rel_desagregada`	23,714 / 22,5	Bloco 3B
`tubos`	41 100	Bloco 4
`ressonancia`	39	Bloco 5A
`desconto_50` / `_75` / `_90` / `_130`	500 / 750 / 800 / 1040	Bloco 5B
`mochila` / `mochila_extras`	142 / 135	Bloco 6
`binpacking`	6	Bloco 7

Referências Bibliográficas

[1] Moreira, A. C. B.; Moreira, M. C. O. *Aula_Programacao_Inteira_Modelagem.ipynb* – GCC118 Programação Matemática. DMM/DCC, Universidade Federal de Lavras, 2026. (Notebook de aula; base desta apostila.)

[2] Toledo, F. M. B.; Costa, A. M. *Programação inteira – Modelagem* (slides de aula, ICMC/USP), com slides do Prof. Celso Carneiro Ribeiro (UFF). Material de apoio de GCC118.

[3] Wolsey, L. A. *Integer Programming*. New York: Wiley, 1998.

[4] Arenales, M.; Armentano, V.; Morabito, R.; Yanasse, H. *Pesquisa Operacional*. Rio de Janeiro: Elsevier/Campus, 2007.

[5] Sierksma, G. *Linear and Integer Programming: Theory and Practice*. New York: Marcel Dekker, 1996.

[6] Bradley, S. P.; Hax, A. C.; Magnanti, T. L. *Applied Mathematical Programming*. Reading: Addison-Wesley, 1977. Cap. 9: *Integer Programming*. Disponível em: `http://web.mit.edu/15.053/www/AMP-Chapter-09.pdf`. Acesso em: out. 2026.

[7] Land, A. H.; Doig, A. G. An automatic method of solving discrete programming problems. *Econometrica*, v. 28, n. 3, p. 497–520, 1960.

[8] INFORMS. *Ailsa H. Land* (perfil biográfico, History of O.R. Excellence). Disponível em: `https://www.informs.org/Explore/History-of-O.R.-Excellence/Biographical-Profiles/Land-Ailsa-H`. Acesso em: out. 2026.

[9] Gomory, R. E. Outline of an algorithm for integer solutions to linear programs. *Bulletin of the American Mathematical Society*, v. 64, n. 5, p. 275–278, 1958.

[10] Dakin, R. J. A tree-search algorithm for mixed integer programming problems. *The Computer Journal*, v. 8, n. 3, p. 250–255, 1965.

[11] Bixby, R. E. A brief history of linear and mixed-integer programming computation. *Documenta Mathematica*, Extra Volume ISMP (Optimization Stories), p. 107–121, 2012.

[12] Koch, T.; Berthold, T.; Pedersen, J.; Vanaret, C. Progress in mathematical programming solvers from 2001 to 2020. *EURO Journal on Computational Optimization*, v. 10, 100031, 2022. Preprint: `https://arxiv.org/abs/2206.09787`.

[13] Vielma, J. P. Mixed integer linear programming formulation techniques. *SIAM Review*, v. 57, n. 1, p. 3–57, 2015. Versão aberta: `https://dspace.mit.edu/handle/1721.1/96480`.

[14] Brown, G. G.; Dell, R. F. Formulating integer linear programs: a rogues' gallery. *INFORMS Transactions on Education*, v. 7, n. 2, p. 153–159, 2007.

[15] Williams, H. P. *Model Building in Mathematical Programming*. 5. ed. Chichester: Wiley, 2013.

[16] Williams, H. P. *Logic and Integer Programming*. New York: Springer, 2009.

[17] Ravelo, S. V. *Programação linear inteira* (aula 22, MC558 – Projeto e Análise de Algoritmos II). Instituto de Computação, Unicamp. Disponível em: `https://ic.unicamp.br/~santiago/assets/mc558/MC558___22.pdf`. Acesso em: out. 2026.

[18] Gurobi Optimization. *Why does Gurobi report my model is infeasible when it has a feasible solution?* Gurobi Help Center. Disponível em: `https://support.gurobi.com/hc/en-us/articles/16565310050833`. Acesso em: out. 2026.

[19] Gurobi Optimization. *Dealing with big-M constraints* (Gurobi Reference Manual, guia de questões numéricas). Disponível em: `https://gurobi.com/documentation/9.0/refman/num_dealing_with_big_m_con.html`. Acesso em: out. 2026.

[20] AMPL Optimization. *Numerical accuracy* (documentação do AMPL MP). Disponível em: `https://mp.ampl.com/modeling-numerics.html`. Acesso em: out. 2026.

[21] Smith, W. E. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, v. 3, n. 1–2, p. 59–66, 1956.

[22] Dantzig, G. B. Discrete-variable extremum problems. *Operations Research*, v. 5, n. 2, p. 266–288, 1957.

[23] Martello, S.; Toth, P. *Knapsack Problems: Algorithms and Computer Implementations*. Chichester: Wiley, 1990.

[24] Kantorovich, L. V. Mathematical methods of organizing and planning production. *Management Science*, v. 6, n. 4, p. 366–422, 1960. (Tradução do original russo de 1939.)

[25] Gilmore, P. C.; Gomory, R. E. A linear programming approach to the cutting-stock problem. *Operations Research*, v. 9, n. 6, p. 849–859, 1961.

[26] Valério de Carvalho, J. M. LP models for bin packing and cutting stock problems. *European Journal of Operational Research*, v. 141, n. 2, p. 253–273, 2002.

[27] Delorme, M.; Iori, M.; Martello, S. Bin packing and cutting stock problems: mathematical models and exact algorithms. *European Journal of Operational Research*, v. 255, n. 1, p. 1–20, 2016.

[28] Bertsimas, D.; Schulz, A. *15.083J Integer Programming and Combinatorial Optimization* (Fall 2009). MIT OpenCourseWare. Disponível em: `https://ocw.mit.edu/courses/15-083j-integer-programming-and-combinatorial-optimization-fall-2009/`. Acesso em: out. 2026.

[29] Mitchell, S.; O'Sullivan, M.; Dunning, I. *PuLP: a linear programming toolkit for Python*. The University of Auckland, 2011. Código e documentação: `https://github.com/coin-or/PuLP`.